



Cours JEE



2021/2022

Nomane NOUAMANE

Nomane.nouamane@iscae.mr



Plan

- Java
- Java EE
 - Architecture d'une application Web
 - Les différentes briques de Java EE
- Environnement de travail
 - Eclipse et serveur Web Tomcat
- Servlet par l'exemple
- Jax-rs par l'exemple
- Angular ou JPA ou les deux



Vous êtes prêts pour Java EE

- Quiz
 - JDK
 - JRE
 - JVM
 - JAR
- Variable d'environnement
 - JAVA_HOME
 - JRE_HOME



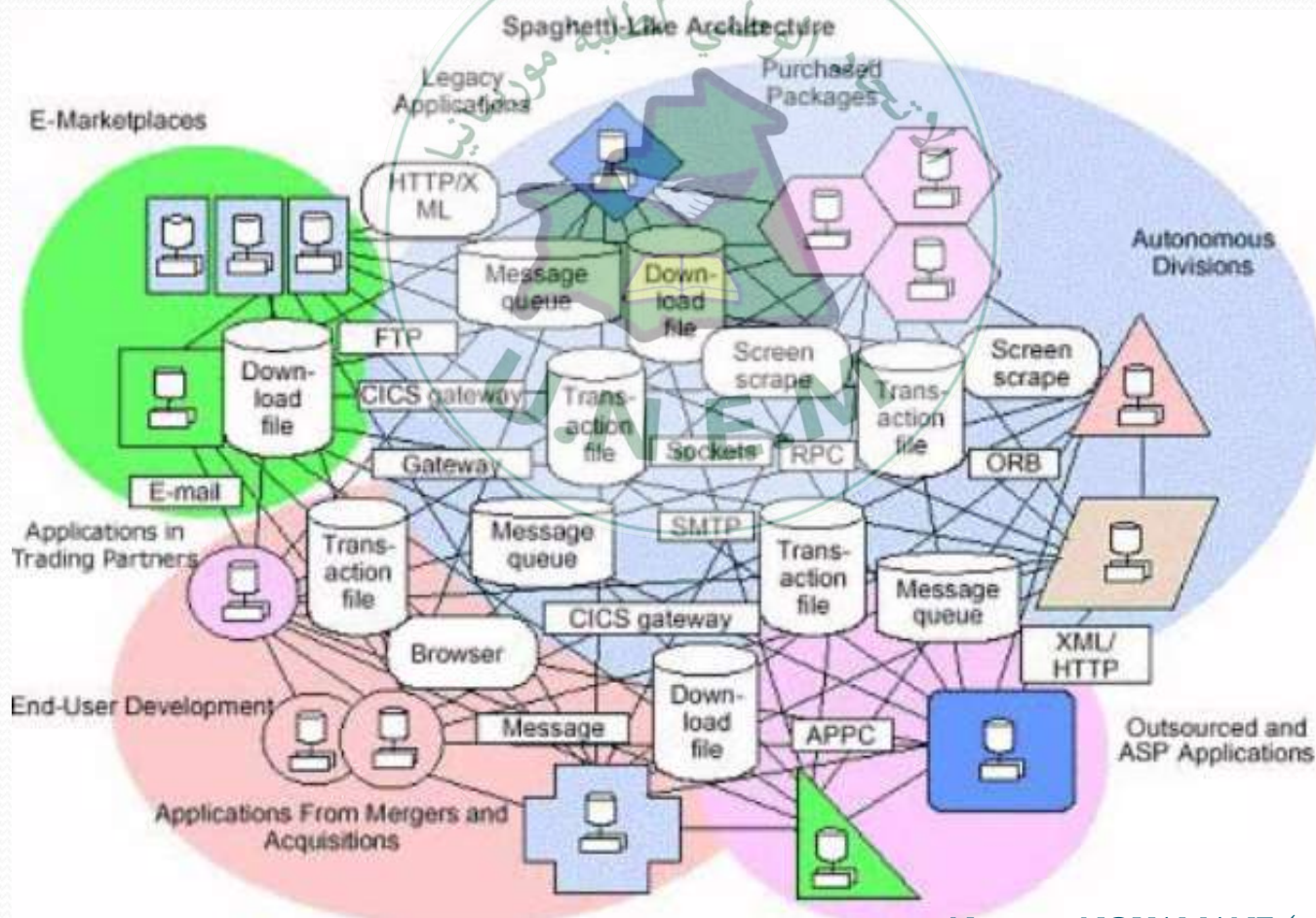


Internet et le Web

- Comment ça marche
- l'internet est le réseau, le support physique de l'information
- les sites internet statiques
- les sites internet dynamiques

Le métier est complexe

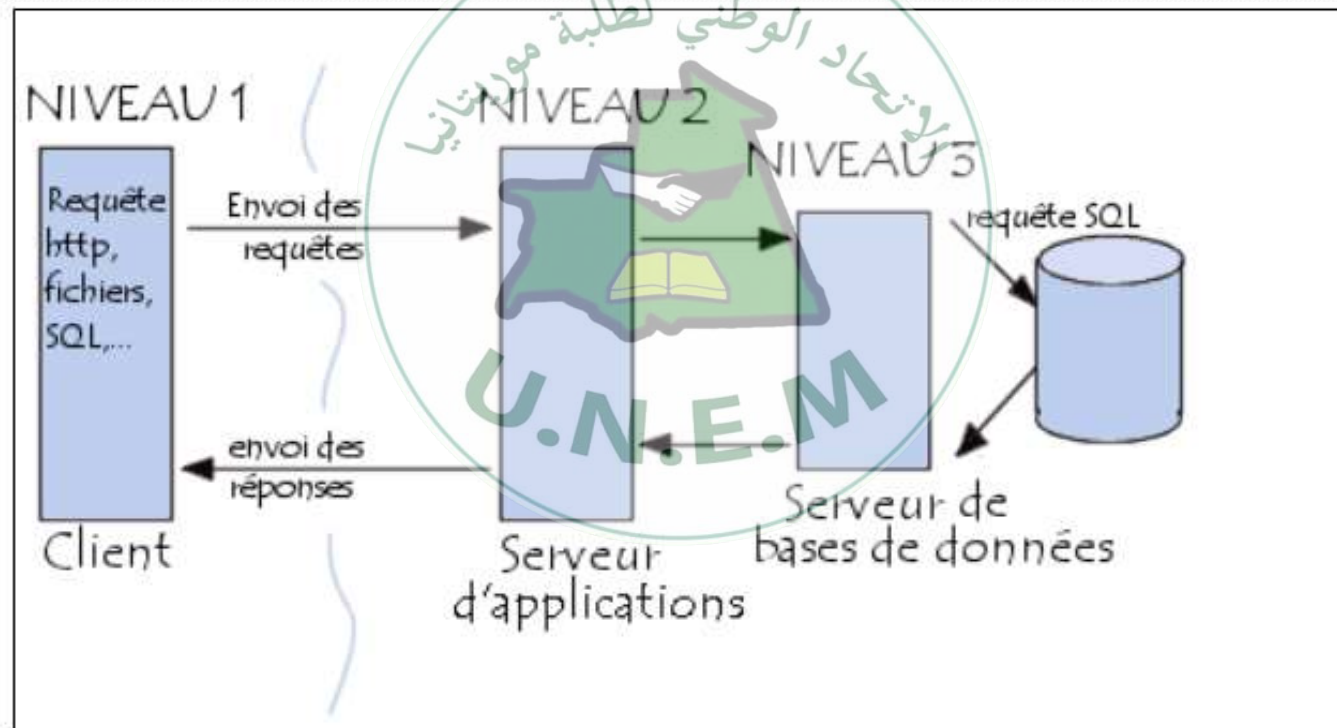
- Application spaghetti



Pour répondre à la complexité

- Diverses solutions possibles
 - Client lourd
 - Nécessite l'installation d'un client spécifique sur le poste de l'utilisateur (swing, java FX ...)
 - Client léger
 - Il suffit d'utiliser le navigateur (Firefox, Chrome...) ou n'importe quel autre client http (curl ...)
- Diverses technologies pour répondre au besoin
 - Java EE, PHP, .NET, Django et Ruby on Rails ...
- Diverses architectures possibles

Architecture multi-tiers





Le fonctionnement

- Parmi toutes les solutions possibles nous avons retenu Java EE! Pourquoi ?
 - Basé sur Java qui est très répandu et largement utilisé
 - Open source
 - Une communauté très active
- Le concurrent direct de Java EE dans le monde Java
 - Spring framework
 - Spring MVC
 - Spring JPA
 - Spring Integration
 - ...



Le fonctionnement

- Un serveur :

Java EE est une extension de la plate-forme standard Java SE, principalement destinée au développement d'applications web.

Application Web - Définition

- Une application Web est n'importe quel logiciel qui :
 - Utilise une technologie Web



- Tourne dans un navigateur

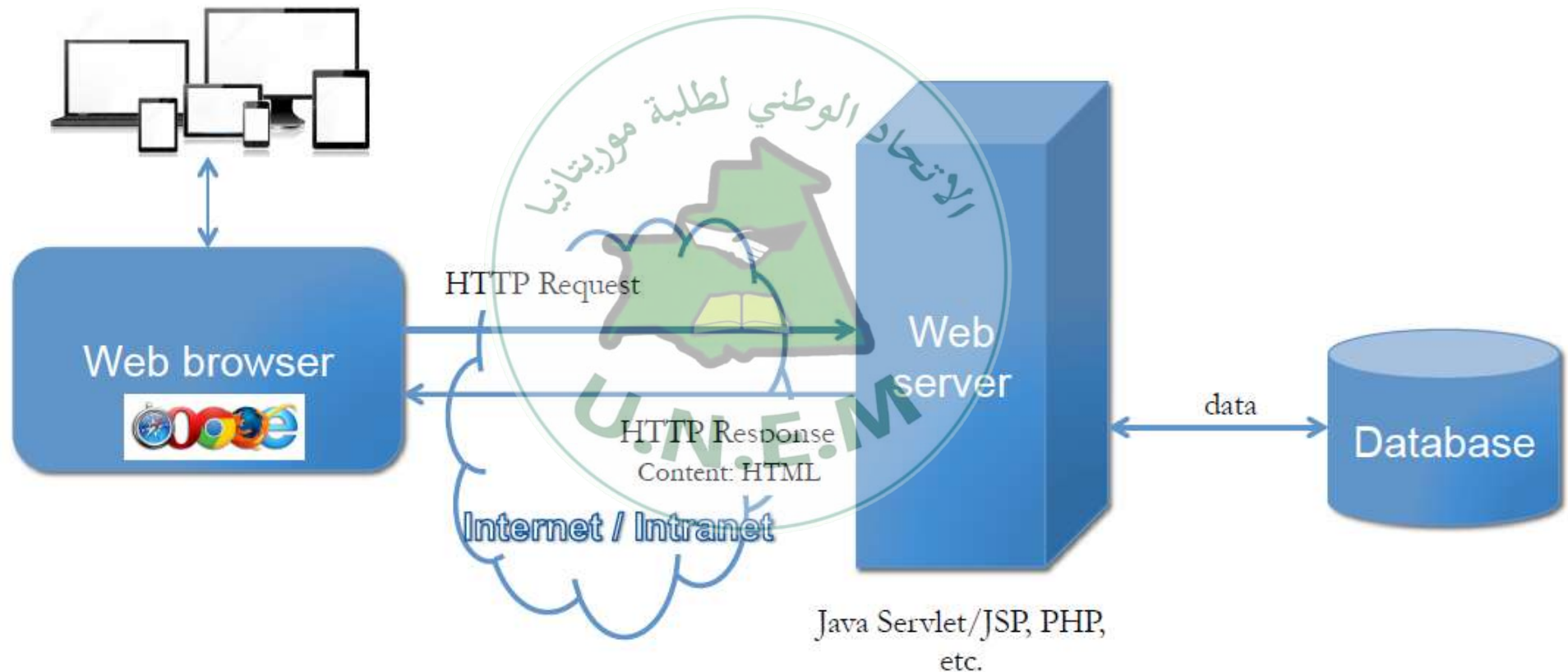


- Offre un certains nombre de fonctionnalités sur un réseau





Architecture basique d'une application Web



Client side

Server side

Application Web

- Navigateur Web (Web browser) :
 - Un **programme** qui affiche les fichiers et les **données** disponible sur **Internet**". [The American Heritage Dictionary]
- Serveur Web (Web server) :
 - Un serveur capable de **traiter** des **requête HTTP** envoyées par un client et envoie une réponse **HTTP**.
- Serveur d'application (Application server) :
 - **Sert** du contenu sur le Web mais aussi il supporte différentes fonctionnalités s (e.g., Messaging services, Security, etc.)



Application Web – étapes d'exécution

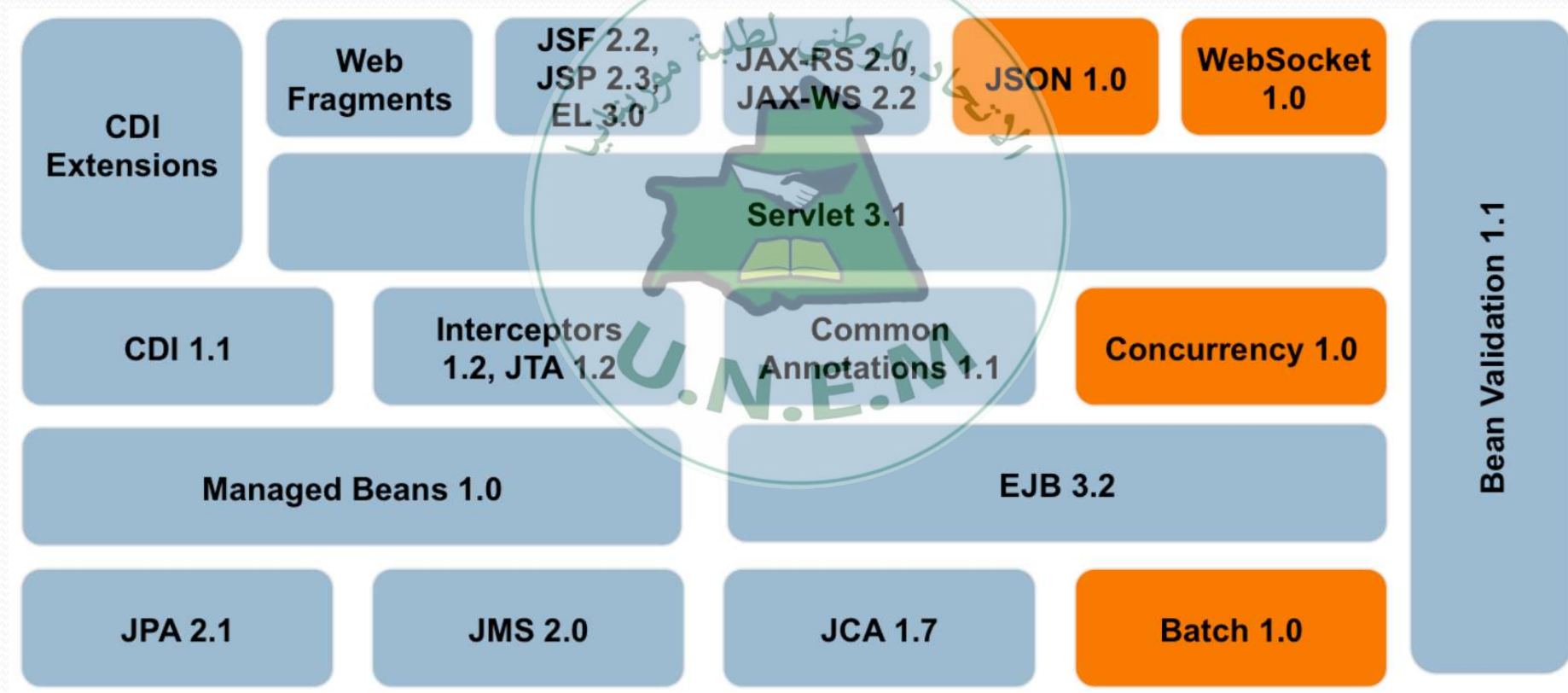
- Lors de l'utilisation des applications Web, on rencontre, généralement, les étapes suivantes :
 - Collecte des données utilisateurs (formulaire, cookies, etc.)
 - Envoie de la requête au serveur (*requête HTTP*)
 - Exécution de la logique métier sur le serveur (*Servlet, PHP, etc.*)
 - Accès à une source de données (*Bases de données*)
 - Collecte des données à envoyer dans la réponse (*réponse HTTP*)
 - Parser et afficher la réponse (*HTML, CSS, etc.*)

Coté
Client

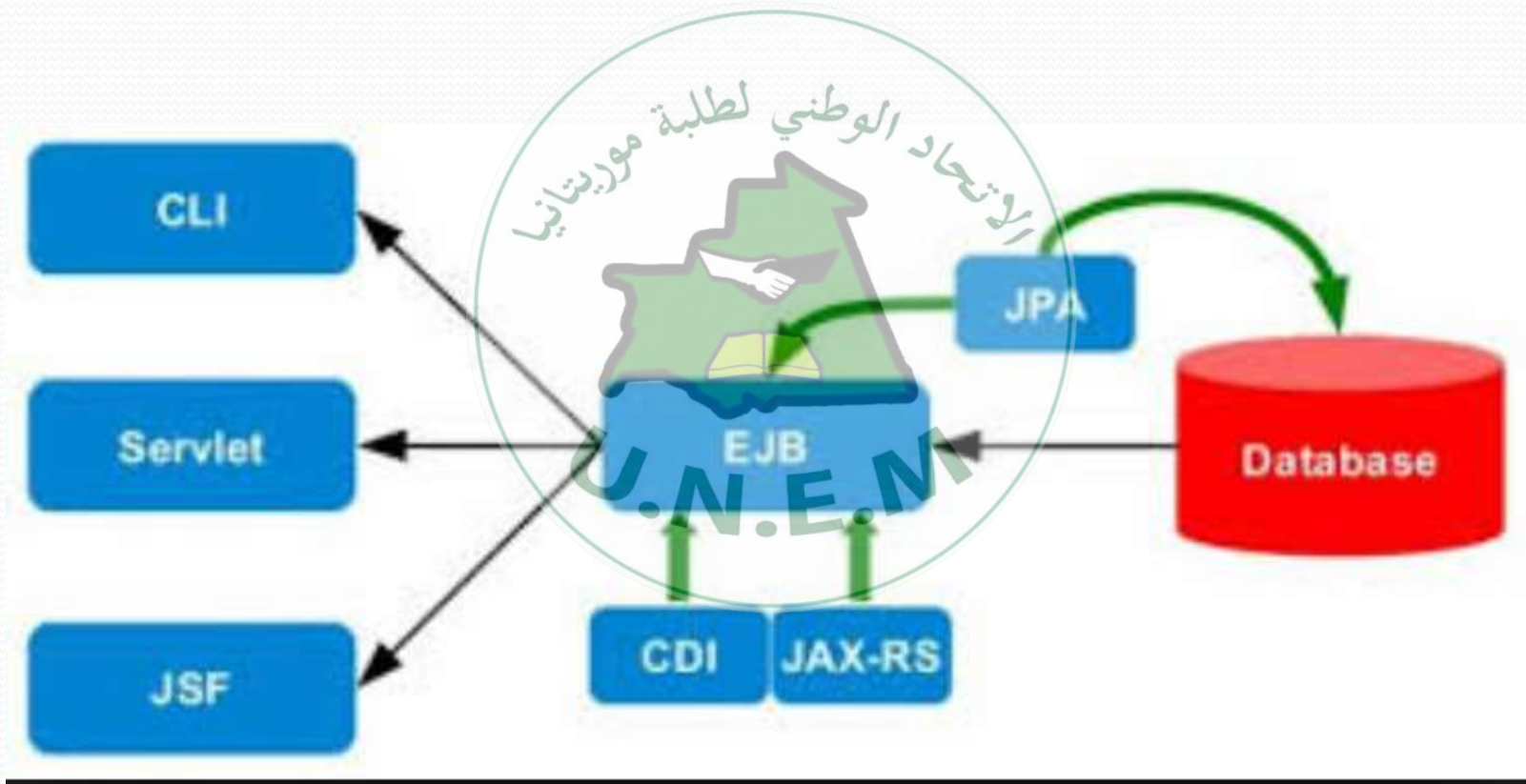
Coté
Serveur

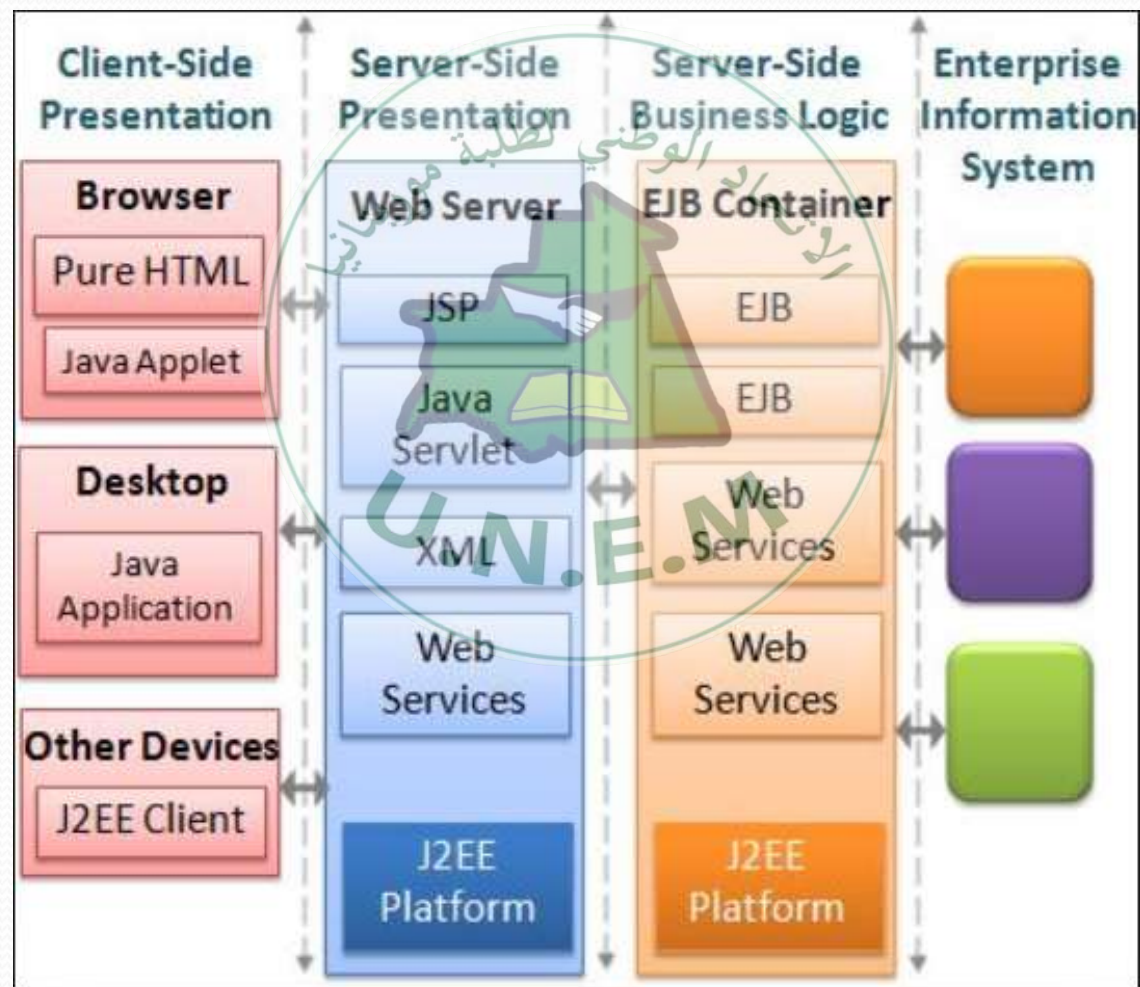
Coté
Client

Ensemble de composant Java EE



Java EE simplifié







Le fonctionnement

- Un client : le navigateur (IE, Firefox, Chrome)
- Un serveur :

C'est un composant que l'on nomme logiquement serveur HTTP. Son travail est simple : il doit écouter tout ce qui arrive sur le port utilisé par le protocole HTTP, le port 80 par exemple, et scruter chaque requête entrante. C'est tout ce qu'il fait, c'est en somme une interface de communication avec le protocole.



Eclipse JEE

- Télécharger la dernière version d'Eclipse pour Java EE
 - eclipse-jee-oxygen-M2-win32-x86_64
- Prérequis d'installation d'Eclipse
 - JDK ou JRE



Téléchargement d'Eclipse

HOME / DOWNLOADS / PACKAGES / ECLIPSE OXYGEN 1A PACKAGES

Eclipse Oxygen 1A Packages

RELEASES

- Oxygen Packages
- Photon Packages
- Neon Packages
- Mars Packages
- Luna Packages
- Kepler Packages
- Juno Packages
- Indigo Packages



Eclipse IDE for Java Developers
180 MB - Downloaded 281,782 Times



Eclipse IDE for Java EE Developers
331 MB - Downloaded 262,008 Times

Windows **32-bit 64-bit**
Mac Cocoa **64-bit**
Linux **32-bit 64-bit**

Windows **32-bit 64-bit**
Mac Cocoa **64-bit**
Linux **32-bit 64-bit**

Tomcat

8.5.23

Please see the [README](#) file for packaging information. It explains what every distribution contains.

Binary Distributions

- Core:
 - [zip \(pgp, md5, sha1\)](#)
 - [tar.gz \(pgp, md5, sha1\)](#)
 - [32-bit Windows zip \(pgp, md5, sha1\)](#)
 - [64-bit Windows zip \(pgp, md5, sha1\)](#)
 - [32-bit/64-bit Windows Service Installer \(pgp, md5, sha1\)](#)
- Full documentation:
 - [tar.gz \(pgp, md5, sha1\)](#)
- Deployer:
 - [zip \(pgp, md5, sha1\)](#)
 - [tar.gz \(pgp, md5, sha1\)](#)
- Extras:
 - [JMX Remote jar \(pgp, md5, sha1\)](#)
 - [Web services jar \(pgp, md5, sha1\)](#)
- Embedded:
 - [tar.gz \(pgp, md5, sha1\)](#)
 - [zip \(pgp, md5, sha1\)](#)



Après la décompression vous aurez le dossier :

> Ce PC > Nouveau nom (D:) > soft > installed > apache-tomcat-8.5.23

Nom	Modifié le	Type	Taille
bin	28/09/2017 12:31	Dossier de fichiers	
conf	28/09/2017 12:31	Dossier de fichiers	
lib	28/09/2017 12:31	Dossier de fichiers	
logs	28/09/2017 12:30	Dossier de fichiers	
temp	28/09/2017 12:31	Dossier de fichiers	
webapps	28/09/2017 12:31	Dossier de fichiers	
work	28/09/2017 12:30	Dossier de fichiers	
LICENSE	28/09/2017 12:31	Fichier	57 Ko
NOTICE	28/09/2017 12:31	Fichier	2 Ko
RELEASE-NOTES	28/09/2017 12:31	Fichier	8 Ko
RUNNING	28/09/2017 12:31	Document texte	17 Ko



- On désigne par :
 - \$CATALINA_HOME est le répertoire d'installation de tomcat. Sur mon ordinateur
`$CATALINA_HOME= D:\soft\installed\apache-tomcat-10.0.27`
- Sur vos ordinateurs vous avez le choix de l'installer dans n'importe quel répertoire

Le répertoire conf contient

- **conf** :

- **server.xml** : contient les éléments de configuration du serveur ;
- **context.xml** : contient les directives communes à toutes les applications web déployées sur le serveur ;
- **tomcat-users.xml** : contient entre autres l'identifiant et le mot de passe permettant d'accéder à l'interface d'administration de votre serveur Tomcat ;
- **web.xml** : contient les paramètres de configuration communs à toutes les applications web déployées sur le serveur

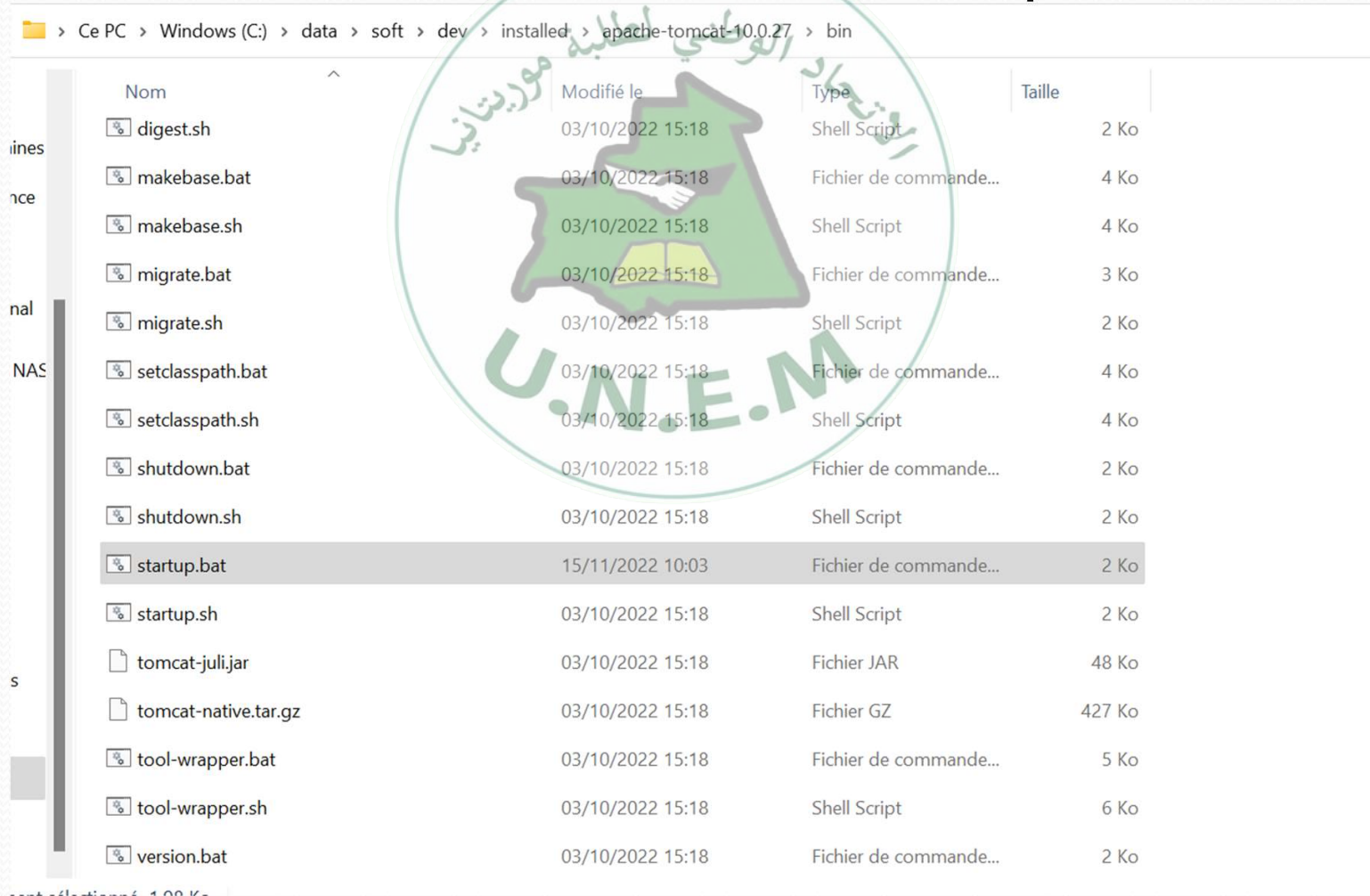
Les autres répertoires

- Bin
 - Contient les scripts permettant de démarrer/arrêter tomcat
- Webapp
 - Le dossier par défaut qui contient les applications déployées sur le serveur
- Logs
 - Contient les traces d'exécution du serveur. Si vous avez un problème consulter ce dossier pour en savoir davantage



premiers pas avec votre serveur tomcat

- Aller dans \$TOMCAT_HOME/bin. Cliquer sur startup



File Explorer path: > Ce PC > Windows (C:) > data > soft > dev > installed > apache-tomcat-10.0.27 > bin

Nom	Modifié le	Type	Taille
digest.sh	03/10/2022 15:18	Shell Script	2 Ko
makebase.bat	03/10/2022 15:18	Fichier de commande...	4 Ko
makebase.sh	03/10/2022 15:18	Shell Script	4 Ko
migrate.bat	03/10/2022 15:18	Fichier de commande...	3 Ko
migrate.sh	03/10/2022 15:18	Shell Script	2 Ko
setclasspath.bat	03/10/2022 15:18	Fichier de commande...	4 Ko
setclasspath.sh	03/10/2022 15:18	Shell Script	4 Ko
shutdown.bat	03/10/2022 15:18	Fichier de commande...	2 Ko
shutdown.sh	03/10/2022 15:18	Shell Script	2 Ko
startup.bat	15/11/2022 10:03	Fichier de commande...	2 Ko
startup.sh	03/10/2022 15:18	Shell Script	2 Ko
tomcat-juli.jar	03/10/2022 15:18	Fichier JAR	48 Ko
tomcat-native.tar.gz	03/10/2022 15:18	Fichier GZ	427 Ko
tool-wrapper.bat	03/10/2022 15:18	Fichier de commande...	5 Ko
tool-wrapper.sh	03/10/2022 15:18	Shell Script	6 Ko
version.bat	03/10/2022 15:18	Fichier de commande...	2 Ko



Une fois le serveur est démarré

- Aller sur un navigateur et taper localhost:8080
- Cliquer sur *Manager App*

Boîte de réception (1 47) x My Drive - Google Drive x Apache Tomcat/8.5.23 x

→ ↻ ⓘ localhost:8080

Home Documentation Configuration Examples Wiki Mailing Lists Find Help

Apache Tomcat/8.5.23

THE APACHE SOFTWARE FOUNDATION
<http://www.apache.org/>

If you're seeing this, you've successfully installed Tomcat. Congratulations!



Recommended Reading:

- [Security Considerations HOW-TO](#)
- [Manager Application HOW-TO](#)
- [Clustering/Session Replication HOW-TO](#)

Server Status
Manager App
Host Manager

Developer Quick Start

Tomcat Setup	Realms & AAA	Examples	Servlet Specifications
First Web Application	JDBC DataSources		Tomcat Versions

Premiers pas avec tomcat

- Un login et un mot de passe vous seront demandés → cliquer sur ok sans saisir de login



401 Unauthorized

You are not authorized to view this page. If you have not changed any configuration files, please examine the file `conf/tomcat-users.xml` in your installation.

For example, to add the `manager-gui` role to a user named `tomcat` with a password of `s3cret`, add the following to the config file listed above.

```
<role rolename="manager-gui"/>
<user username="tomcat" password="s3cret" roles="manager-gui"/>
```

Note that for Tomcat 7 onwards, the roles required to use the manager application were changed from the single `manager` role to the following four roles. You v

- `manager-gui` - allows access to the HTML GUI and the status pages
- `manager-script` - allows access to the text interface and the status pages
- `manager-jmx` - allows access to the JMX proxy and the status pages
- `manager-status` - allows access to the status pages only

The HTML interface is protected against CSRF but the text and JMX interfaces are not. To maintain the CSRF protection:

- Users with the `manager-gui` role should not be granted either the `manager-script` or `manager-jmx` roles.
- If the text or jmx interfaces are accessed through a browser (e.g. for testing since these interfaces are intended for tools not humans) then the browser i

For more information - please see the [Manager App HOW-TO](#).

Nomane NOUAMANE (nomane12@gmail.com)



Interpréter les erreurs

Le message est très explicite. Il nous indique que pour avoir accès à l'interface *Manger App* il va falloir qu'il y ait un utilisateur que l'on doit déclarer dans `$CATALINA_HOME/conf/tomcat-users.xml`.
L'utilisateur en question **doit** avoir le role *manager-gui*

- Allons ouvrir le fichier `$CATALINA_HOME/conf/tomcat-users.xml`

```
<!--  
  <role rolename="tomcat"/>  
  <role rolename="role1"/>  
  <user username="tomcat" password="<must-be-changed>" roles="tomcat"/>  
  <user username="both" password="<must-be-changed>" roles="tomcat,role1"/>  
  <user username="role1" password="<must-be-changed>" roles="role1"/>  
-->  
</tomcat-users>
```

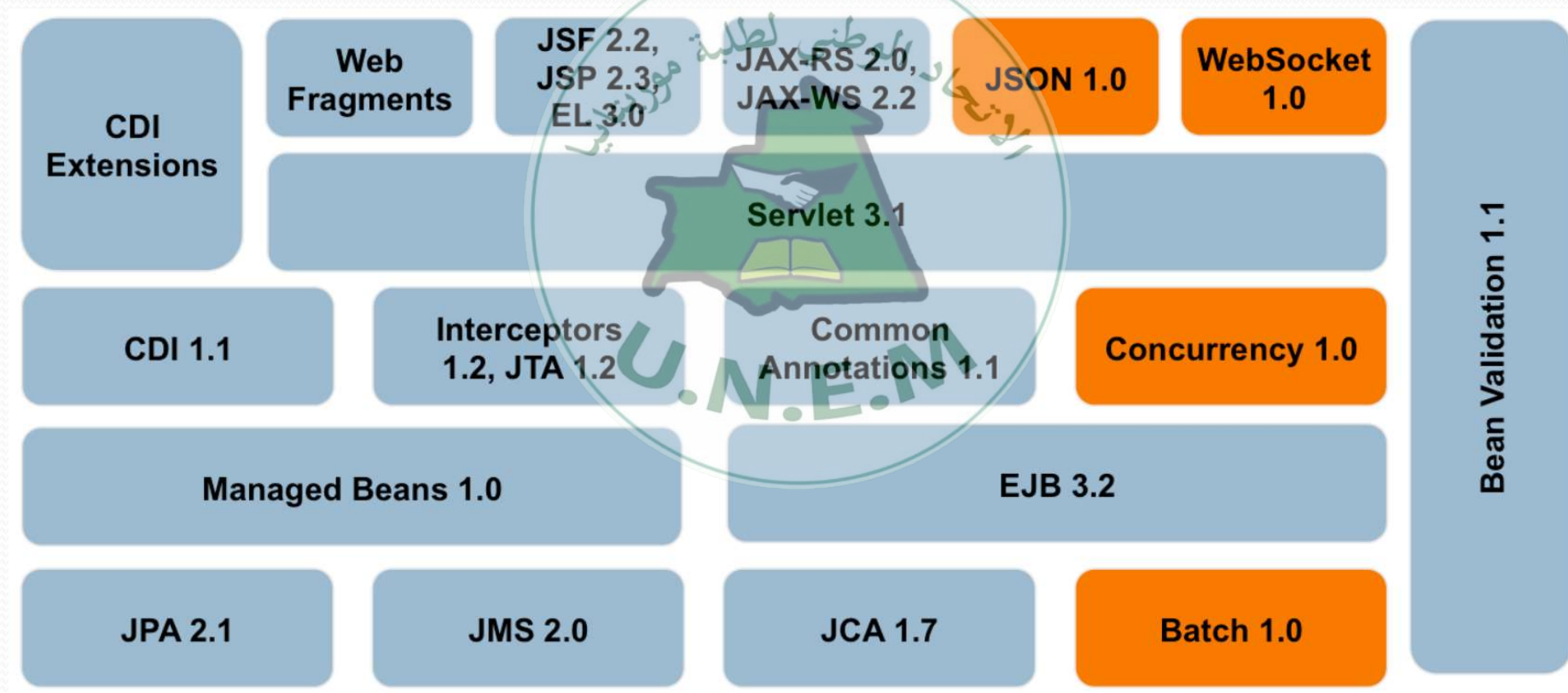
- On doit enlever les « <!-- » et «--> » qui précède et suit la section des user

```
<role rolename="tomcat"/>
<role rolename="role1"/>
<user username="tomcat" password="tomcat" roles="manager-gui"/>
<user username="both" password="toto" roles="tomcat,role1"/>
<user username="role1" password="tata" roles="role1"/>

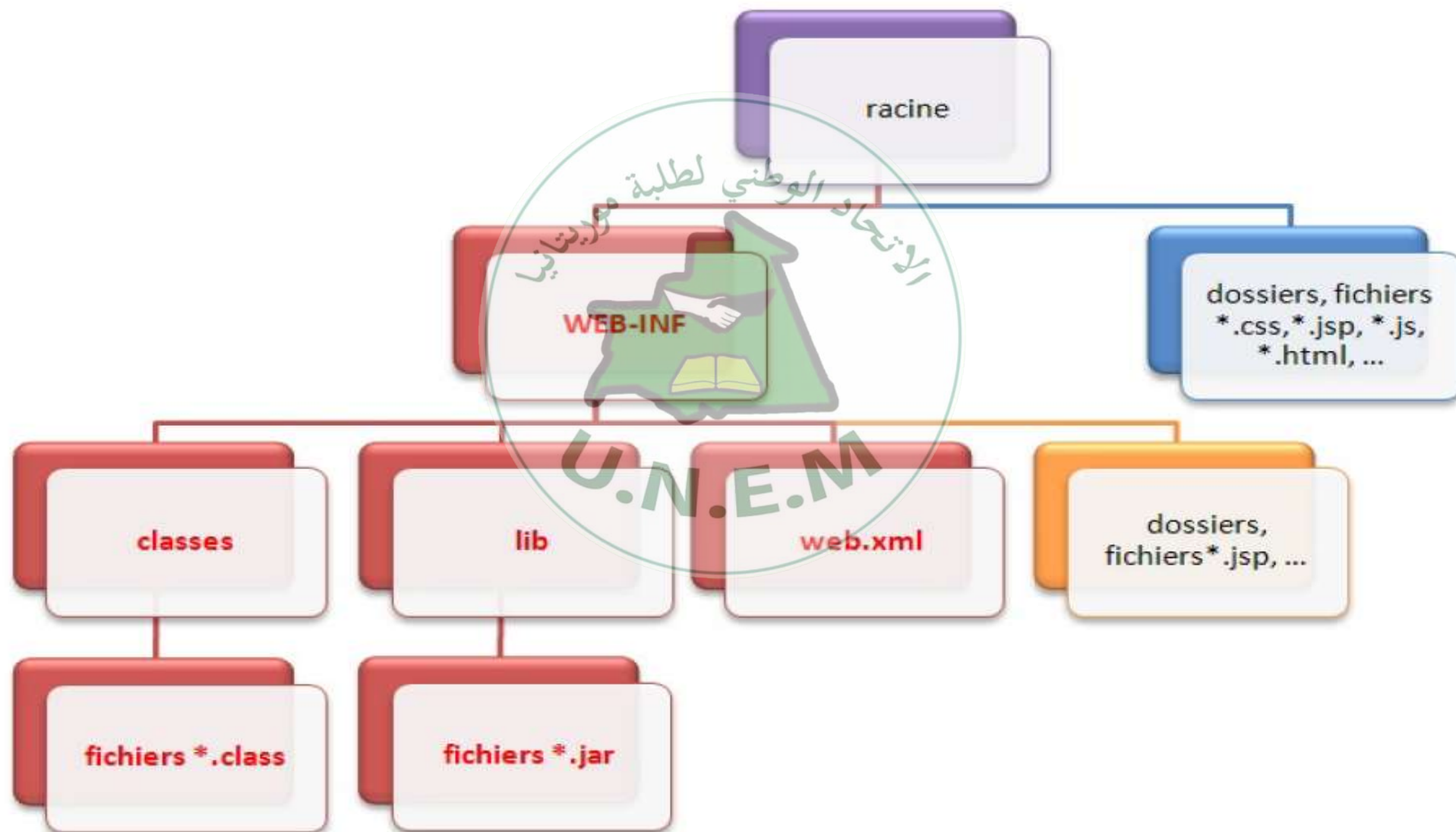
</tomcat-users>
```

- Pour ces modifications prennent effet il va falloir arrêter et redémarrer Tomcat
 - Cliquez sur \$CATALINA_HOME/bin/shutdown pour arrêter
 - Et cliquez sur \$CATALINA_HOME/bin/startup pour le démarrer
- Essayer de nouveau d'accéder à Manager App (localhost:8080)

Ensemble de composant Java EE



Structure standard d'un projet Java EE



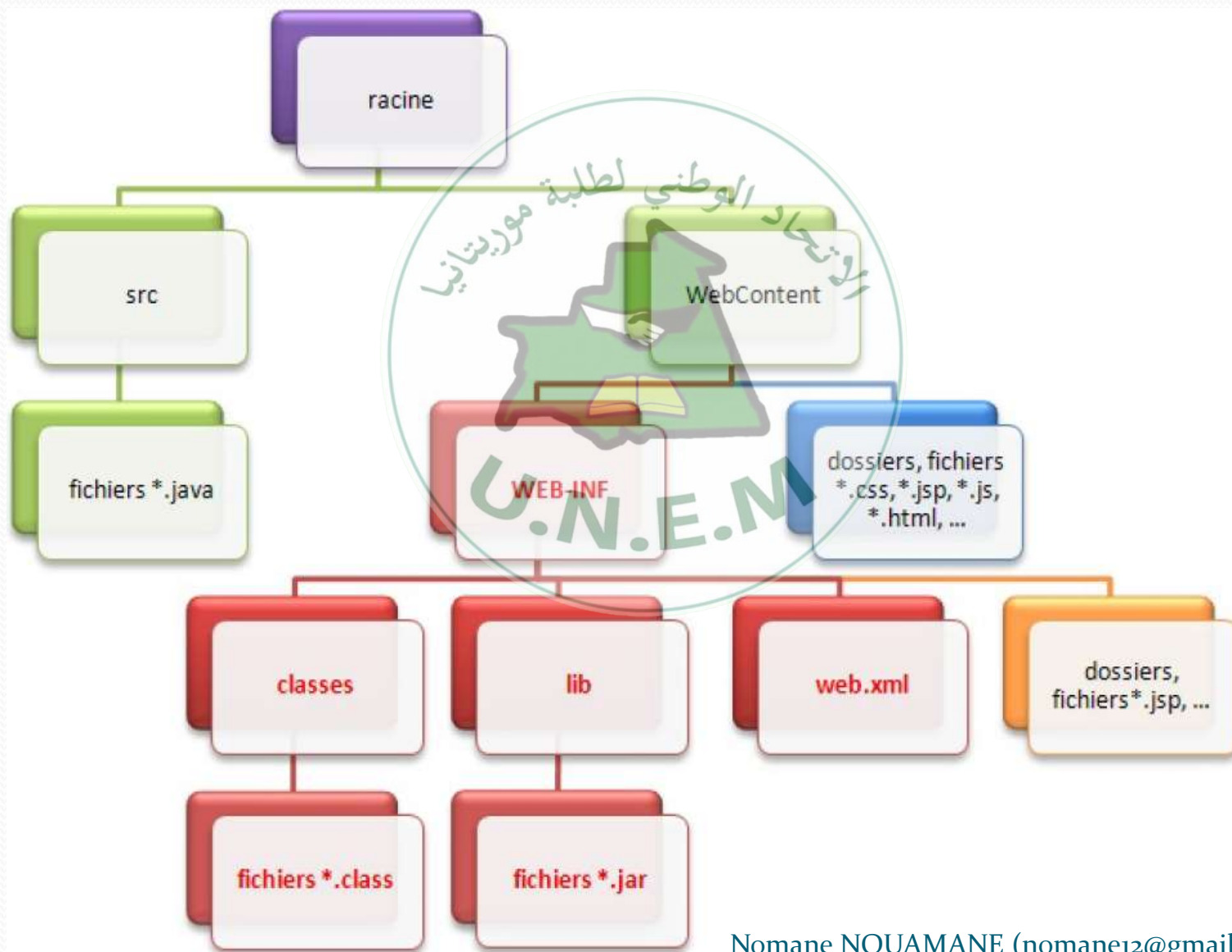


Structure standard d'un projet Java EE

Toute application web Java EE doit respecter une structure de dossiers standard, qui est définie dans les spécifications de la plateforme

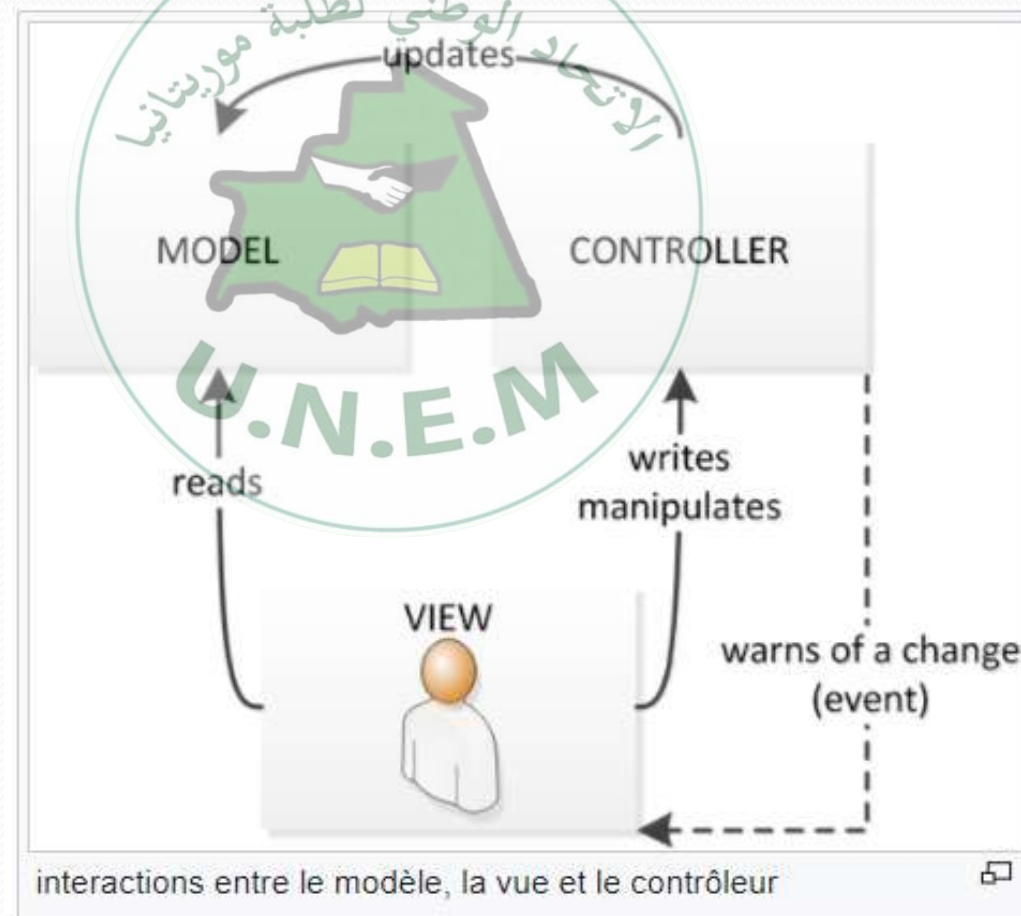
Attention à Eclipse qui modifie la structure standard et adopte la structure suivante

Structure d'une application Java EE sous Eclipse



MVC (Modèle-vue-contrôleur)

- Un modèle (Model) contient les données à afficher.
- Une vue (View) contient la présentation de l'interface graphique.
- Un contrôleur (Controller) contient la logique concernant les actions effectuées par l'utilisateur.



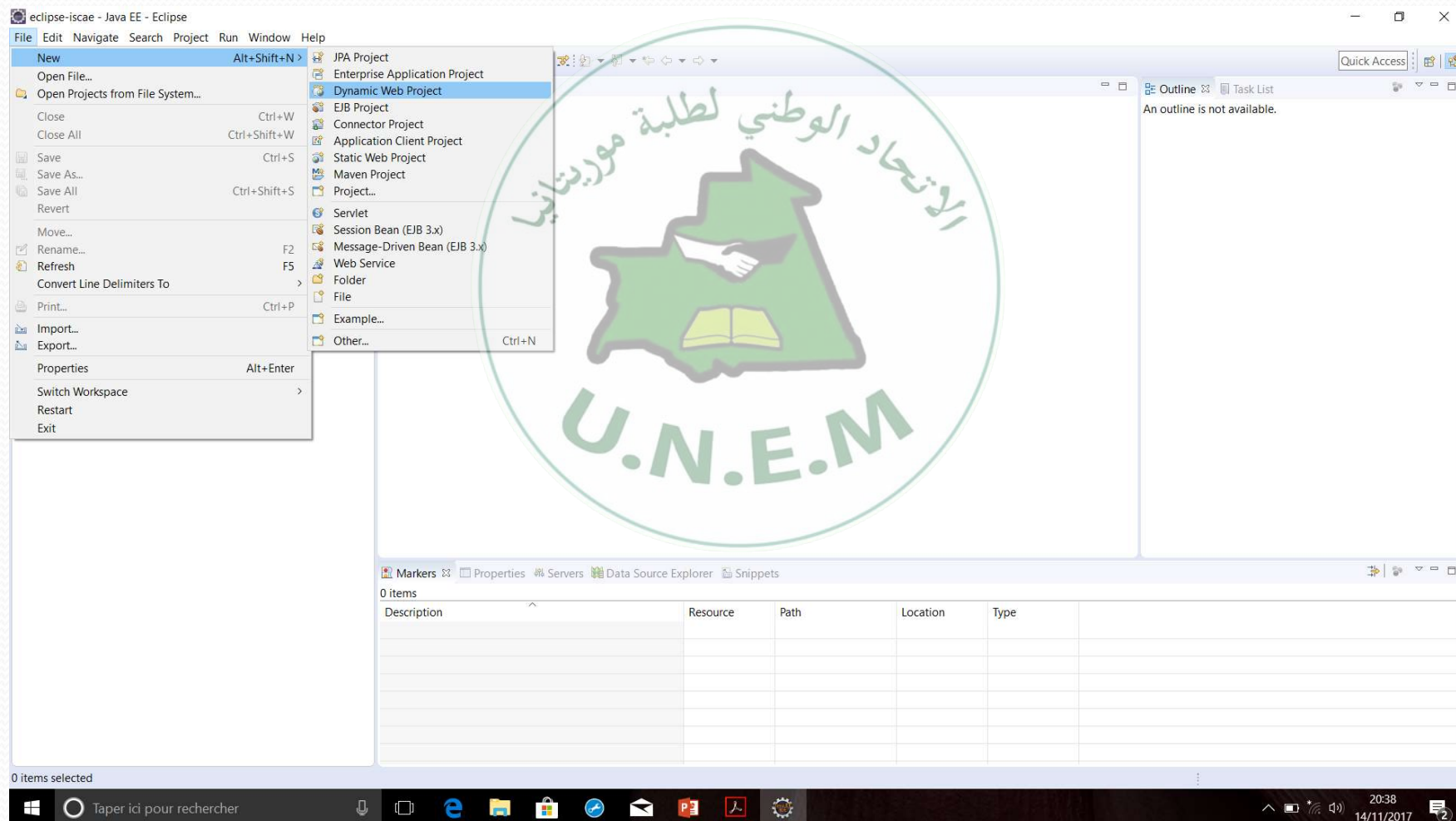


Servlet

- Le client envoie une requête HTTP
- Côté serveur Web (tomcat), la requête sera interceptée par la servlet qui joue le rôle du gendarme
 - La servlet aiguille la requête vers le programme responsable du traitement à effectuer
 - Par rapport au modèle MVC
 - La servlet sera utilisée plutôt dans la partie *Cotroller*



- Les étapes graphiques pour créer et déployer un projet





New Dynamic Web Project

Dynamic Web Project

Create a standalone Dynamic Web project or add it to a new or existing Enterprise Application.

Project name:

Project location

☒ Use default location

Location:

Target runtime

Dynamic web module version

Configuration

The default configuration provides a good starting point. Additional facets can later be installed to add new functionality to the project.

EAR membership

☐ Add project to an EAR

EAR project name:

Working sets

☐ Add project to working sets

Working sets:



Configure un serveur Tomcat

New Dynamic Web Project

Dynamic Web Project

Create a standalone Dynamic Web project or add it to a new or existing Enterprise Application.

Project name:

Project location

☒ Use default location

Location:

Target runtime

Dynamic web module version

Configuration

The default configuration provides a good starting point. Additional facets can later be installed to add new functionality to the project.

EAR membership

☐ Add project to an EAR

EAR project name:

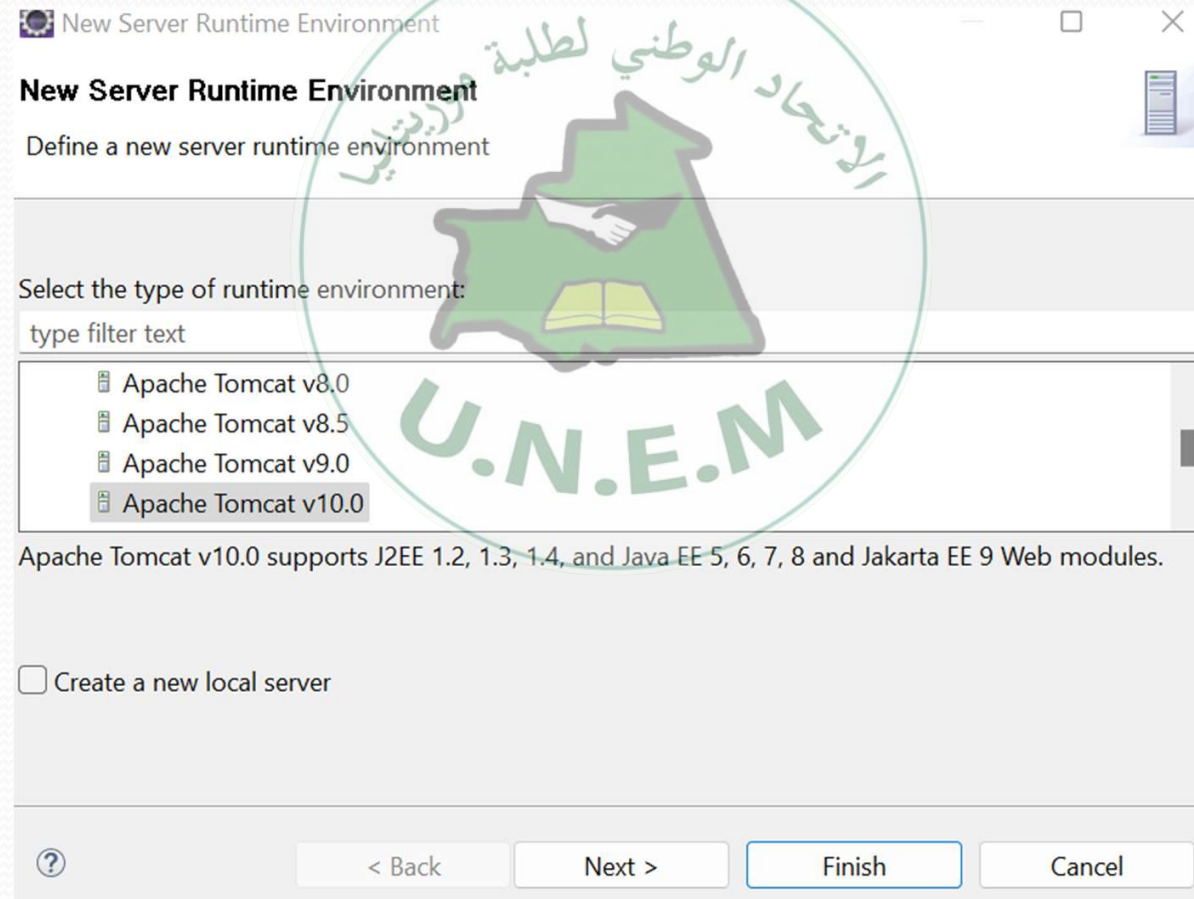
Working sets

☐ Add project to working sets

Working sets:



Configurer la version de Tomcat





enseigner le chemin de Tomcat et cliquer sur Finish

New Server Runtime Environment
Define a new server runtime environment

Select the type of runtime environment:

type filter text

- Apache Tomcat v8.0
- Apache Tomcat v8.5
- Apache Tomcat v9.0
- Apache Tomcat v10.0

Apache Tomcat v10.0 supports J2EE 1.2, 1.3, 1.4, and Java EE 5, 6, 7, 8 and Jakarta EE 9 Web modules.

☐ Create a new local server



Server



Specify the installation directory

Name:

Apache Tomcat v10.0 (2)

Tomcat installation directory:

C:\data\soft\dev\installed\apache-tomcat-10.0.27

Browse...

Download and Install...

JRE:

Workbench default JRE



Installed JREs...

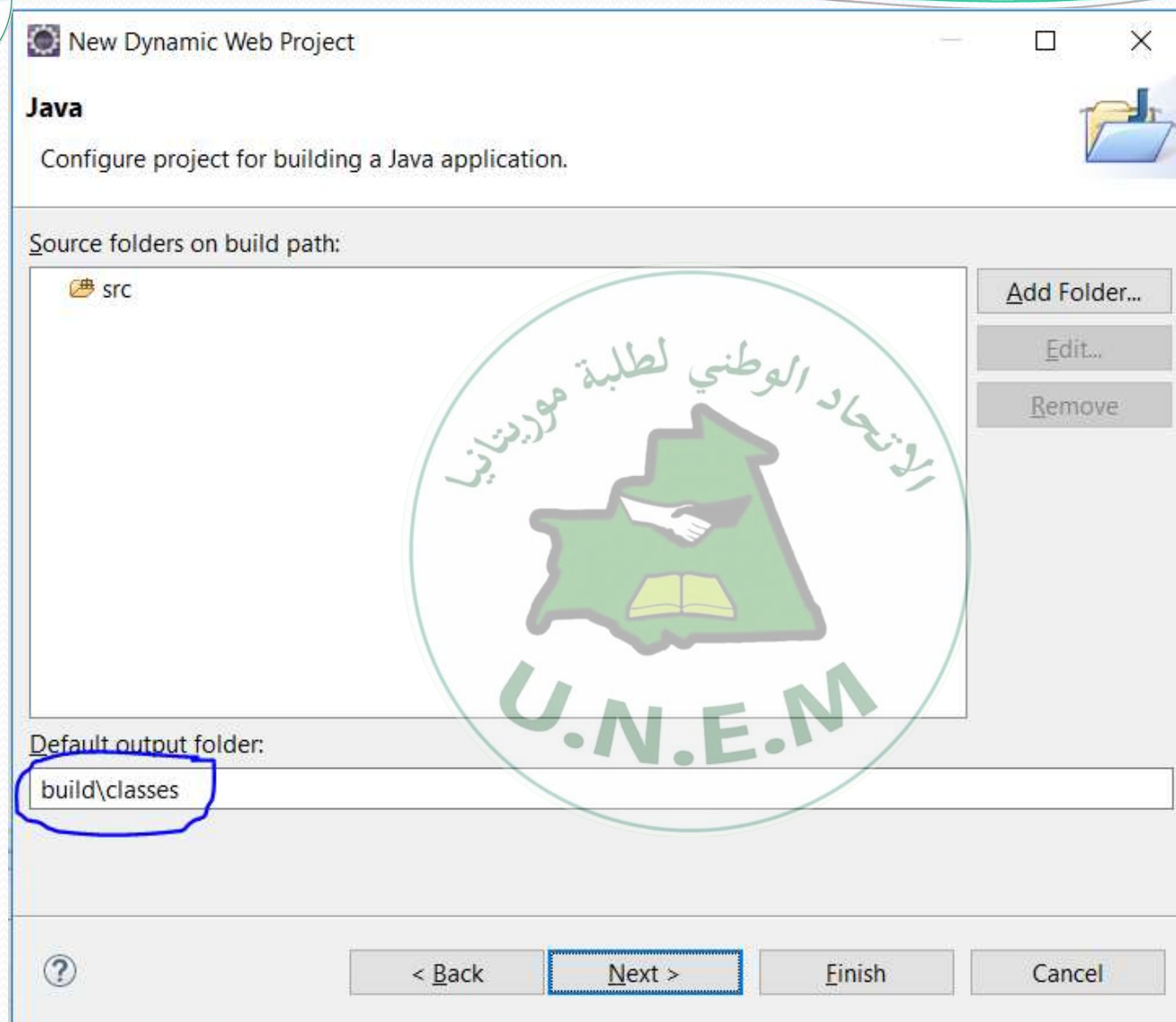


< Back

Next >

Finish

Cancel



- le chemin du build est à changer plus tard



Cocher la case Generate deployment descriptor

New Dynamic Web Project

Web Module
Configure web module settings.

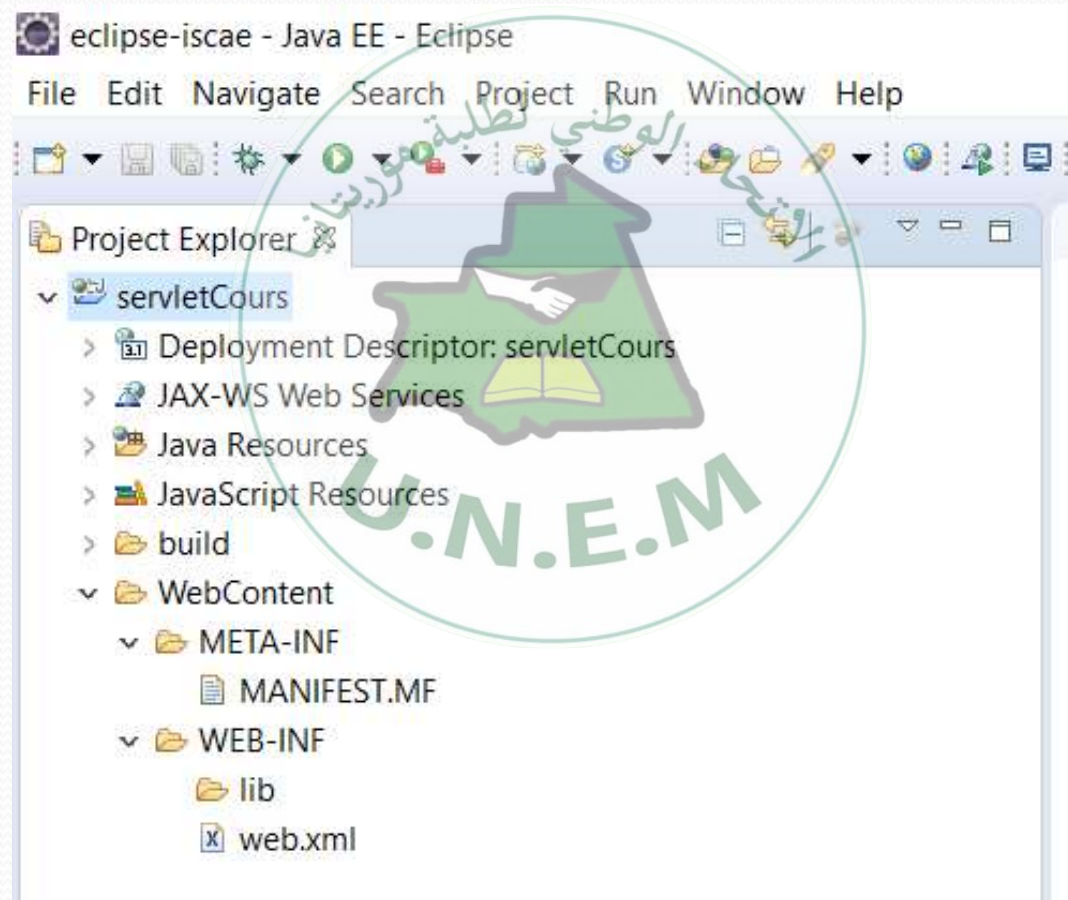
Context root:

Content directory:

☒ Generate web.xml deployment descriptor

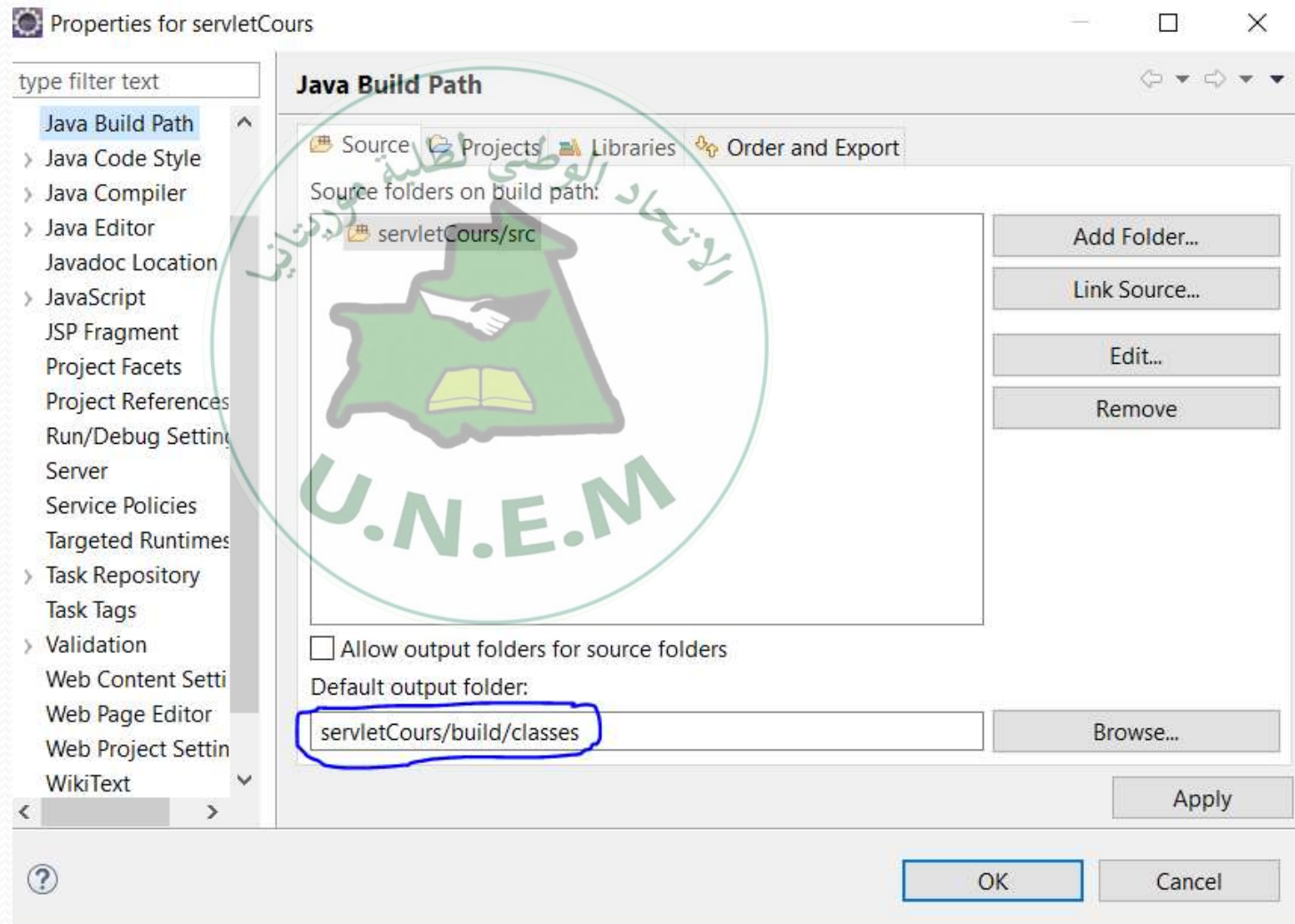


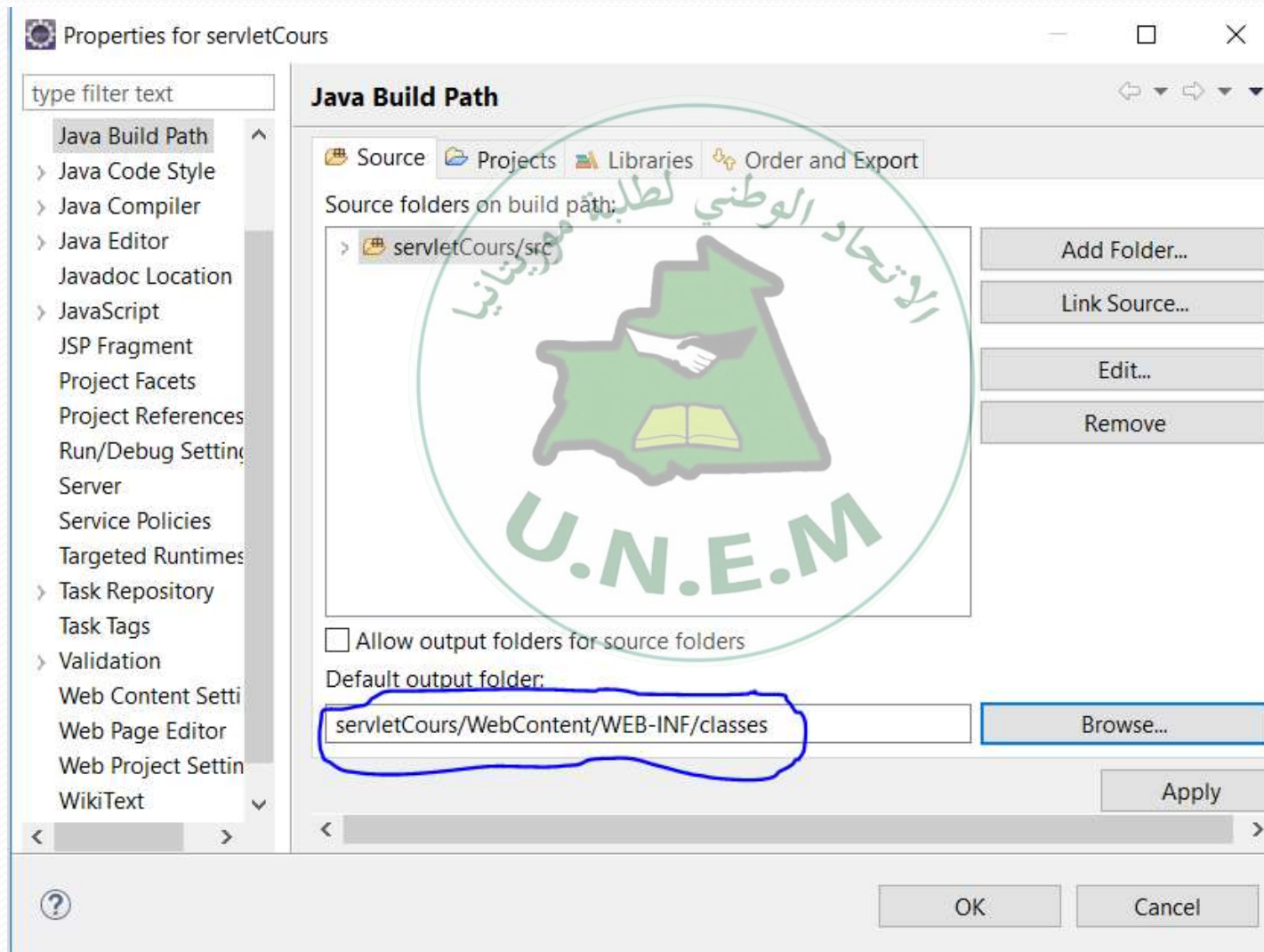
Votre projet ressemblera à ça





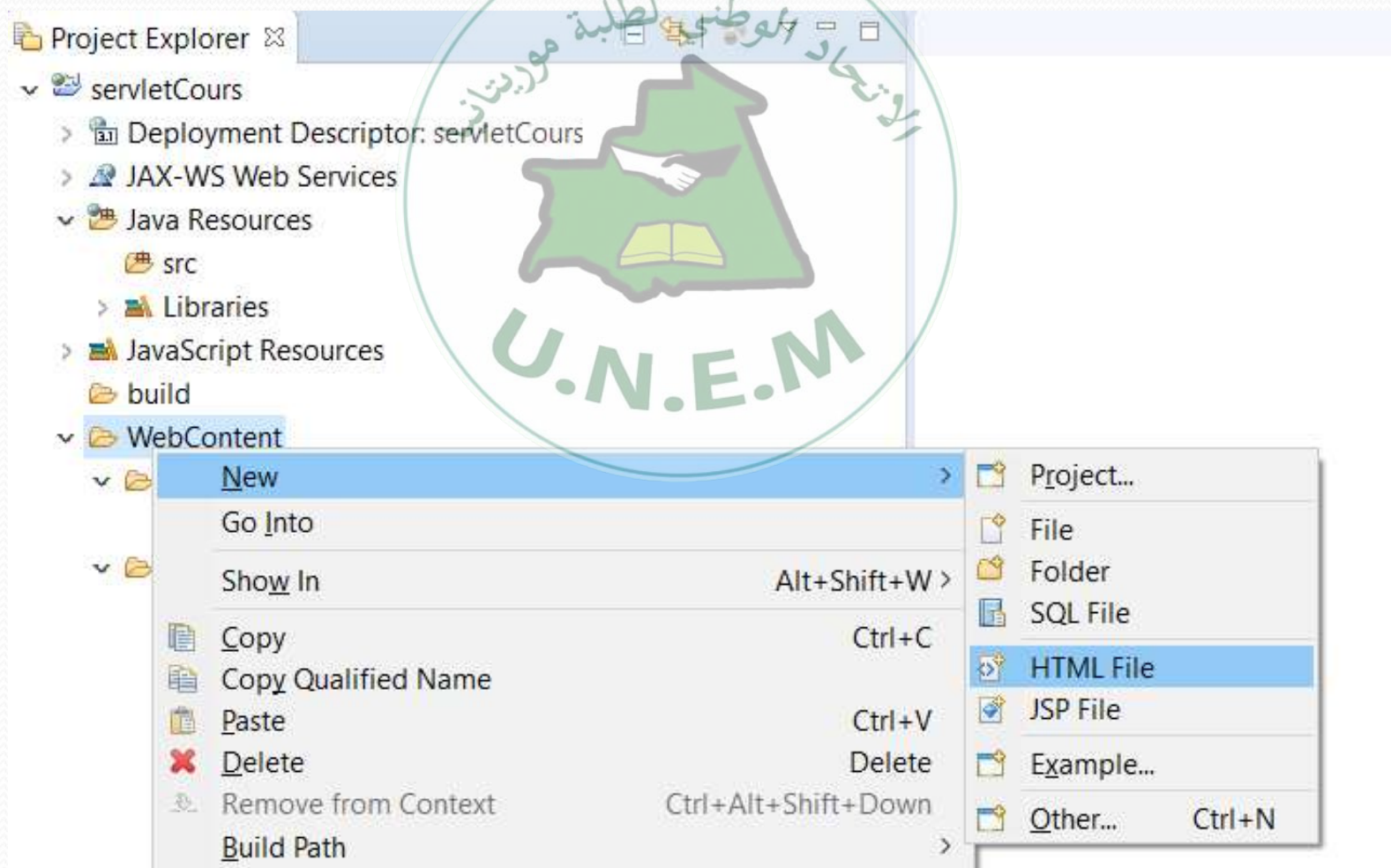
Changer le chemin du build path du projet. Cliquer bouton droit sur le projet → properties → Java build path





Créons notre premier exemple

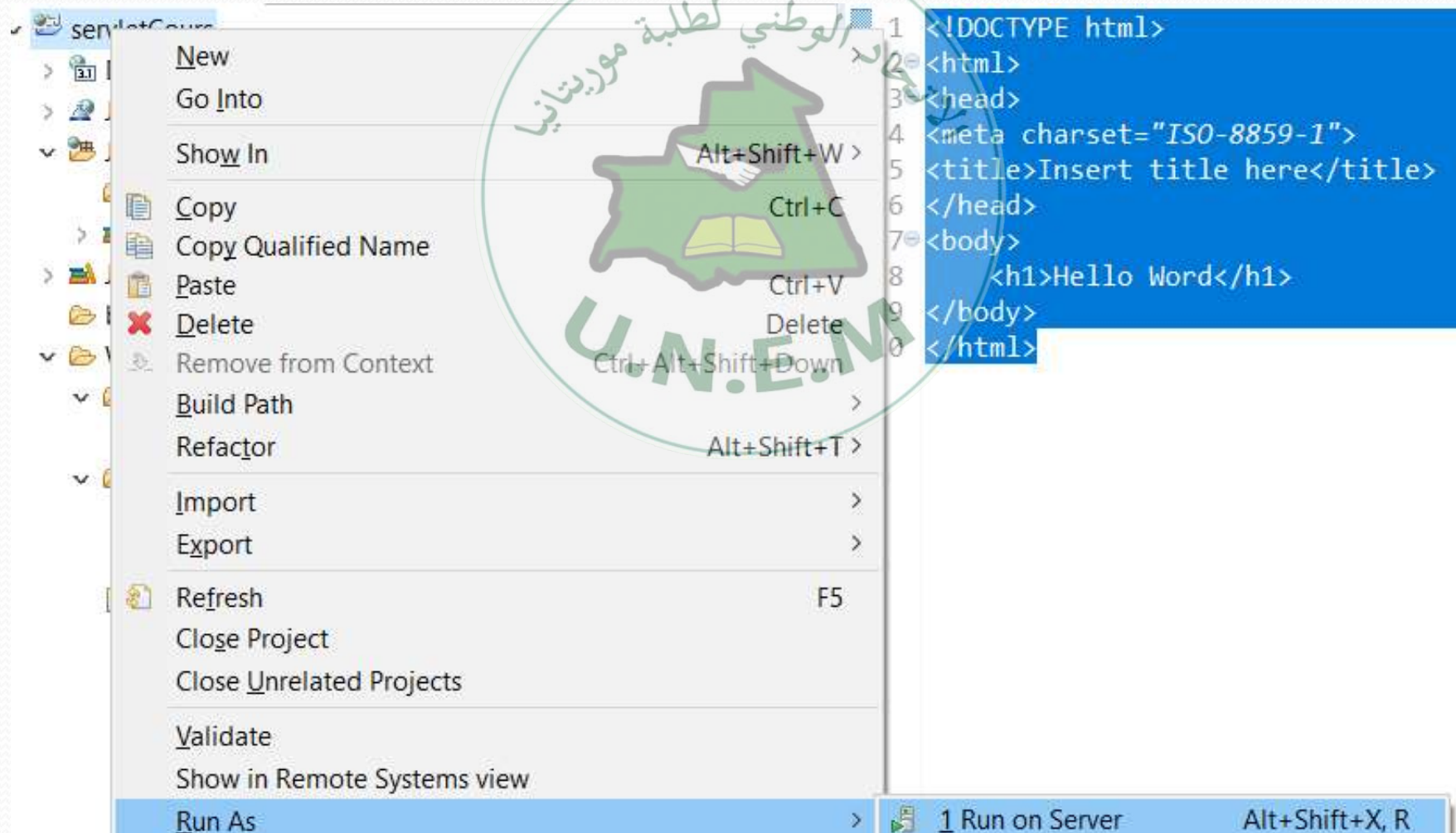
- Créer un fichier HTML nommé index.html dans WebContent





```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="ISO-8859-1">
    <title>Insert title here</title>
  </head>
  <body>
    <h1>Hello Word</h1>
  </body>
</html>
```

- Bouton droit sur le projet → Run As → Run on Server





Cliquer sur Finish

Run On Server

Select which server to use

How do you want to select the server?

☐ Choose an existing server

☒ Manually define a new server

Select the server type:

type filter text

- Tomcat v8.0 Server
- Tomcat v8.5 Server
- Tomcat v9.0 Server

Publishes and runs J2EE and Java EE Web projects and server configurations to a local Tomcat server.

Server's host name: localhost

Server name: Tomcat v8.5 Server at localhost

Server runtime environment: Apache Tomcat v8.5 [Add...](#)

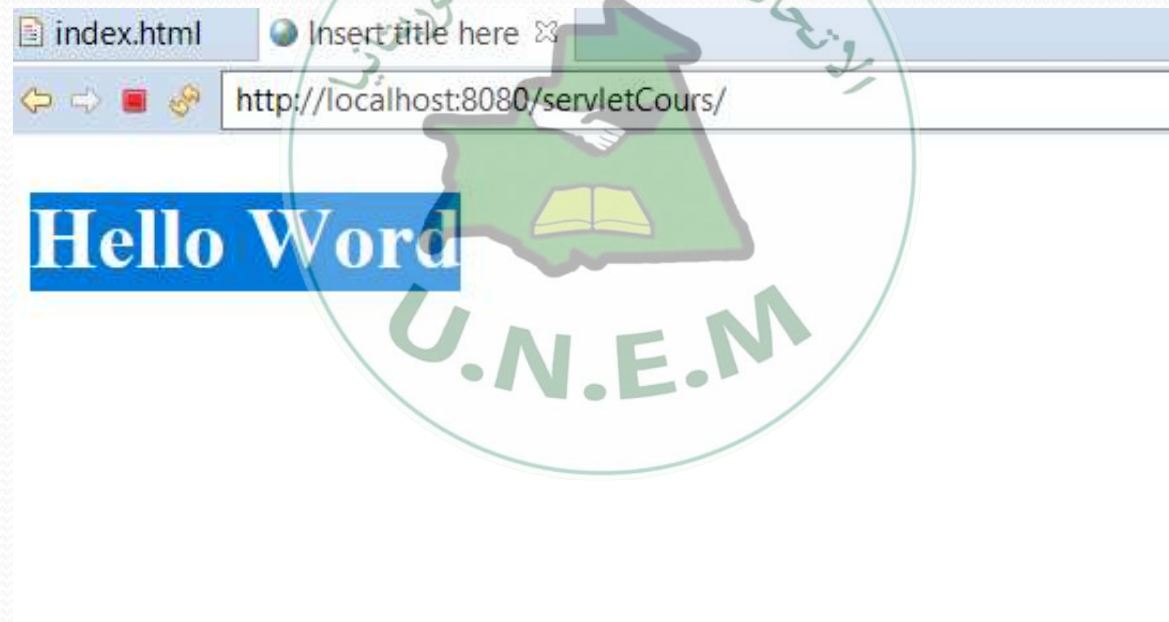
[Configure runtime environments...](#)

☐ Always use this server when running this project

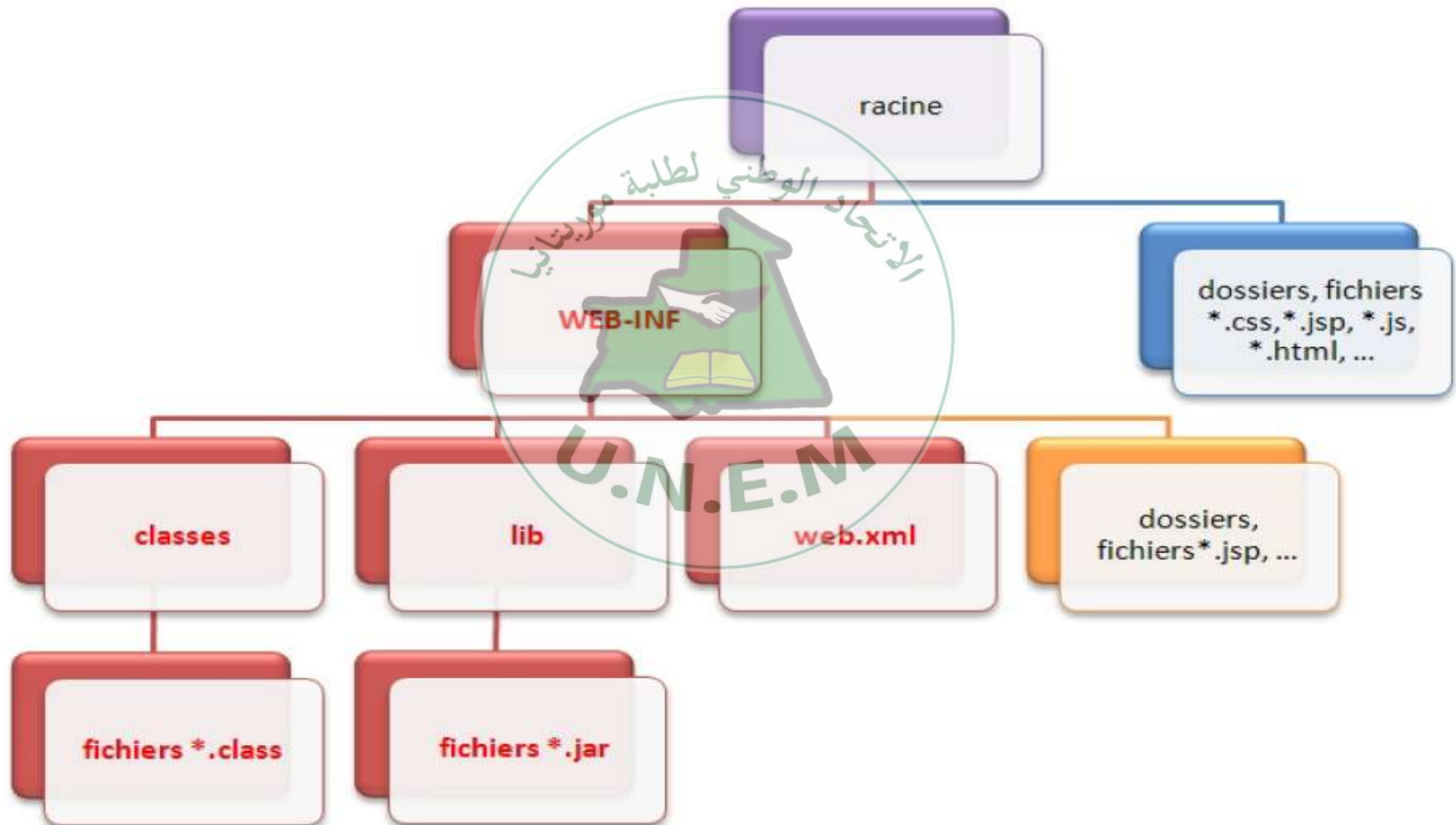
[?](#) [< Back](#) [Next >](#) **Finish** [Cancel](#)



Le résultat



Structure standard d'un projet Java EE



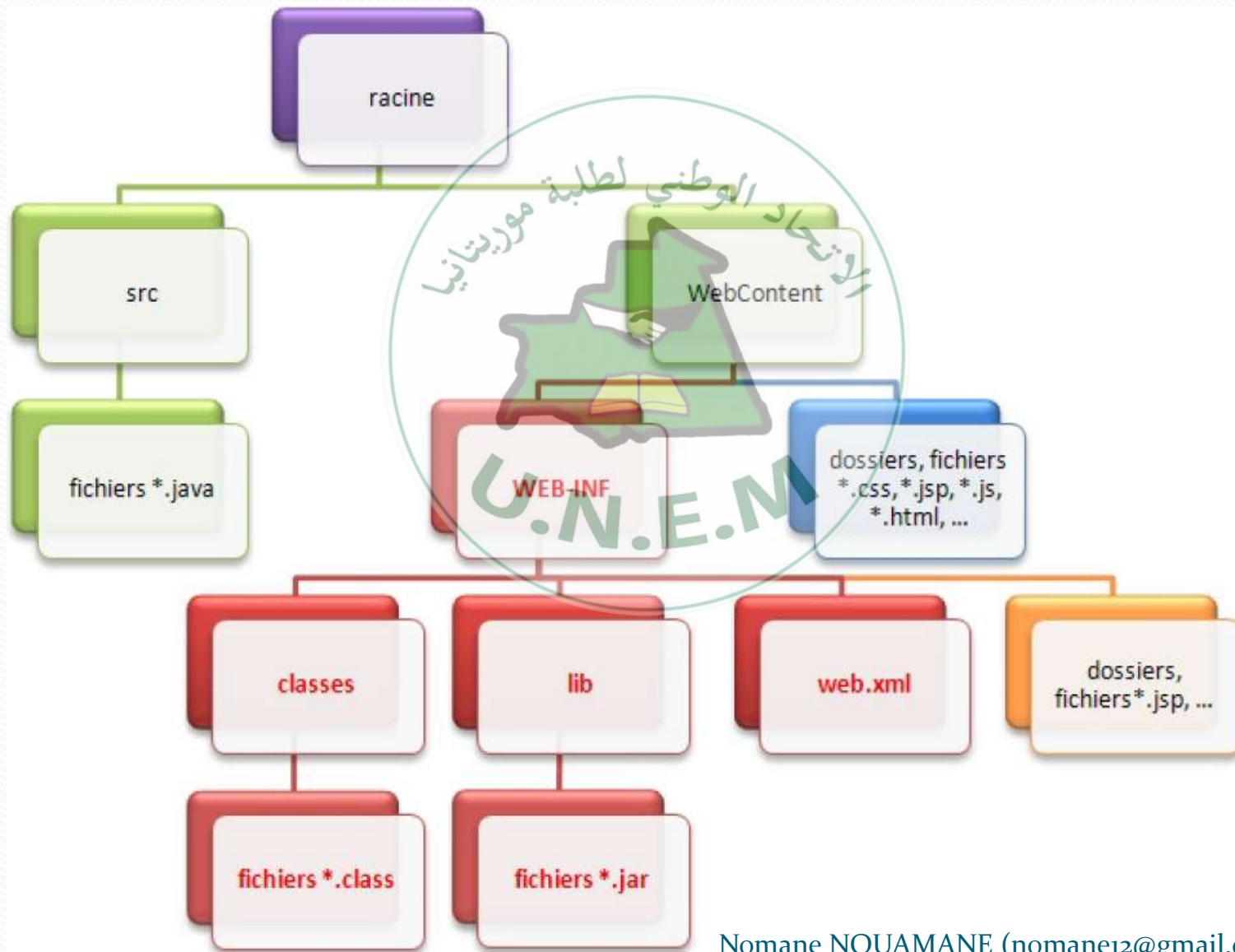


Structure standard d'un projet Java EE

Toute application web Java EE doit respecter une structure de dossiers standard, qui est définie dans les spécifications de la plateforme

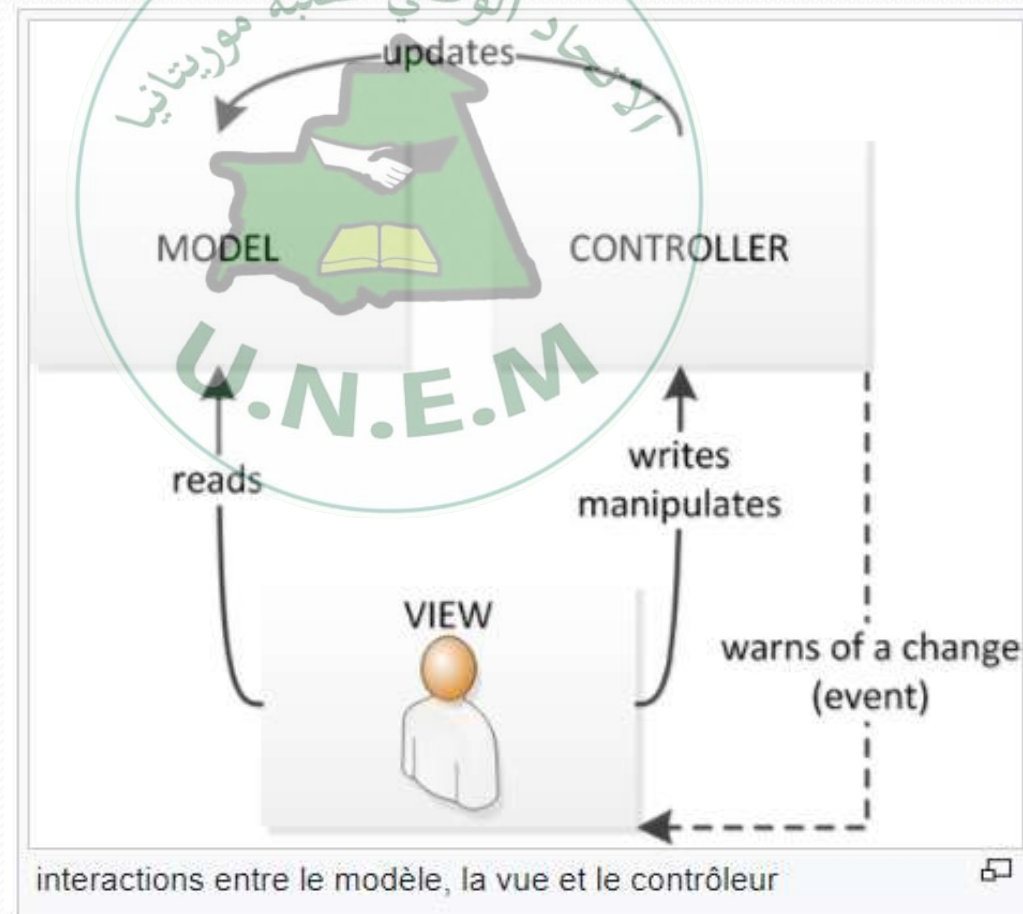
Attention à Eclipse qui modifie la structure standard et adopte la structure suivante

Structure d'une application Java EE sous Eclipse



MVC (Modèle-vue-contrôleur)

- Un modèle (Model) contient les données à afficher.
- Une vue (View) contient la présentation de l'interface graphique.
- Un contrôleur (Controller) contient la logique concernant les actions effectuées par l'utilisateur.

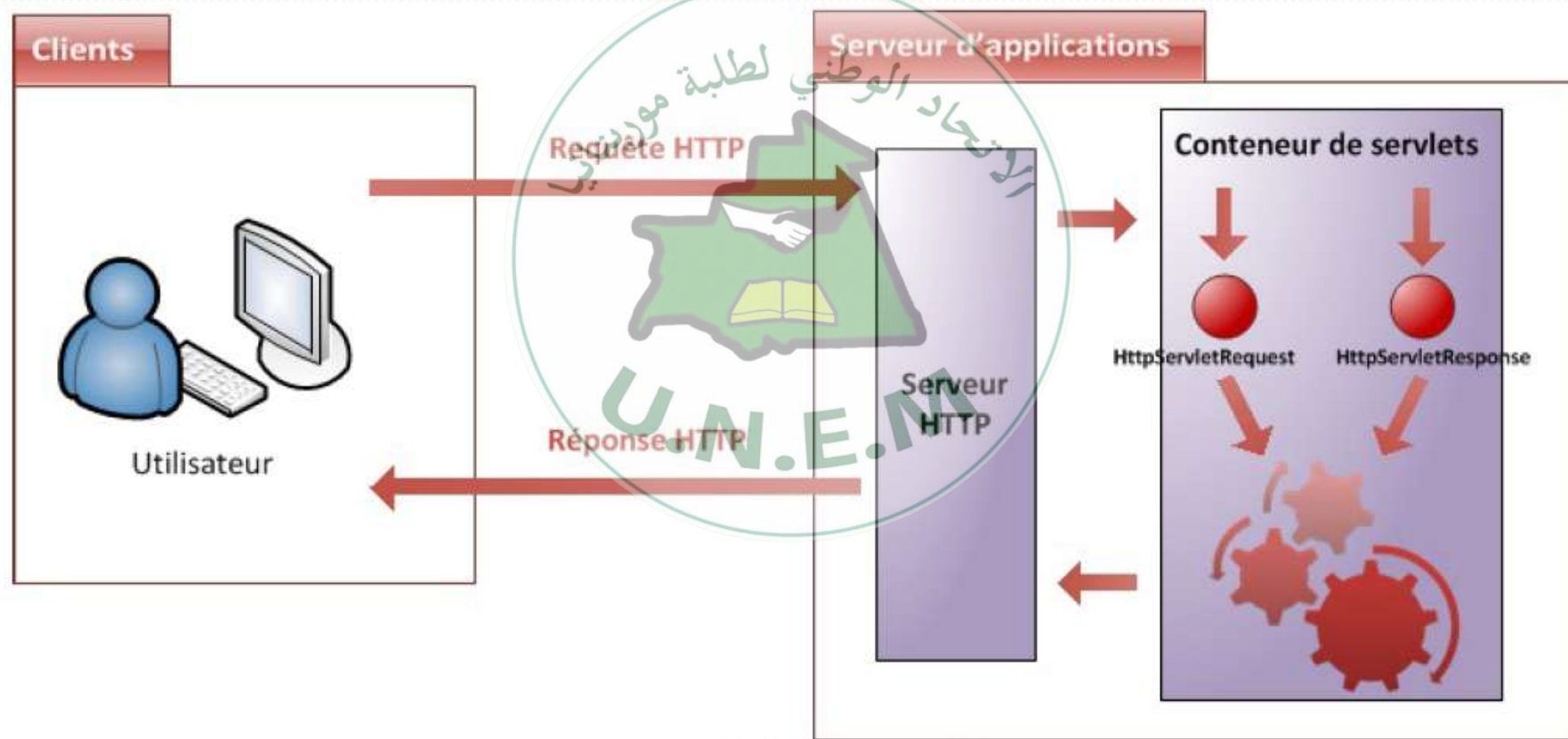




Servlet

- Le client envoie une requête HTTP
- Coté serveur Web (tomcat), la requête sera interceptée par la servlet qui joue le rôle du gendarme
 - La servlet aiguille la requête vers le programme responsable du traitement à effectuer
 - Par rapport au modèle MVC
 - La servlet sera utilisée plutôt dans la partie *Cotroller*

Servlet



Servlet

Une servlet est en réalité une simple classe Java, qui a la particularité de **permettre le traitement de requêtes et la personnalisation de réponses**. Pour faire simple, dans la très grande majorité des cas une servlet n'est rien d'autre qu'une **classe** capable de recevoir une requête HTTP envoyée depuis le **navigateur** de l'utilisateur, et de lui renvoyer une réponse HTTP.

- HttpServlet a deux paramètres :
 - HttpServletRequest : cet objet contient la requête HTTP, et donne accès à toutes ses informations, telles que les en-têtes (*headers*) et le corps de la requête.
 - HttpServletResponse : cet objet initialise la réponse HTTP qui sera renvoyée au client, et permet de la personnaliser, en initialisant par exemple les en-têtes et le corps (nous verrons comment par la suite).



Principe de fonctionnement des Servlets

- Les servlets sont donc des programmes Java qui tournent sur le serveur Web pour "principalement" construire dynamiquement des pages Web.
- La construction dynamique de pages Web est utile pour plusieurs raisons :
 - **La page dépend des données utilisateur.** Par exemple, les moteurs de recherches, les site marchand utilisent la construction dynamique de leur contenu pages
 - **Le contenu change fréquemment.** Par exemple, site météo, sites de news.



Structure de base d'une servlet

```
// Un squelette de définition de servlet HTTP
// importation de packages
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;

// définition de la classe
public class ServletFoo extends HttpServlet {
    public void init(...) {
        // initialisation des variables d'état de la servlet
    }

    public void service(...) {
        // Définition des traitements de la servlet
    }

    public void destroy() {
        // Libération des ressources éventuelles
    }
}
```



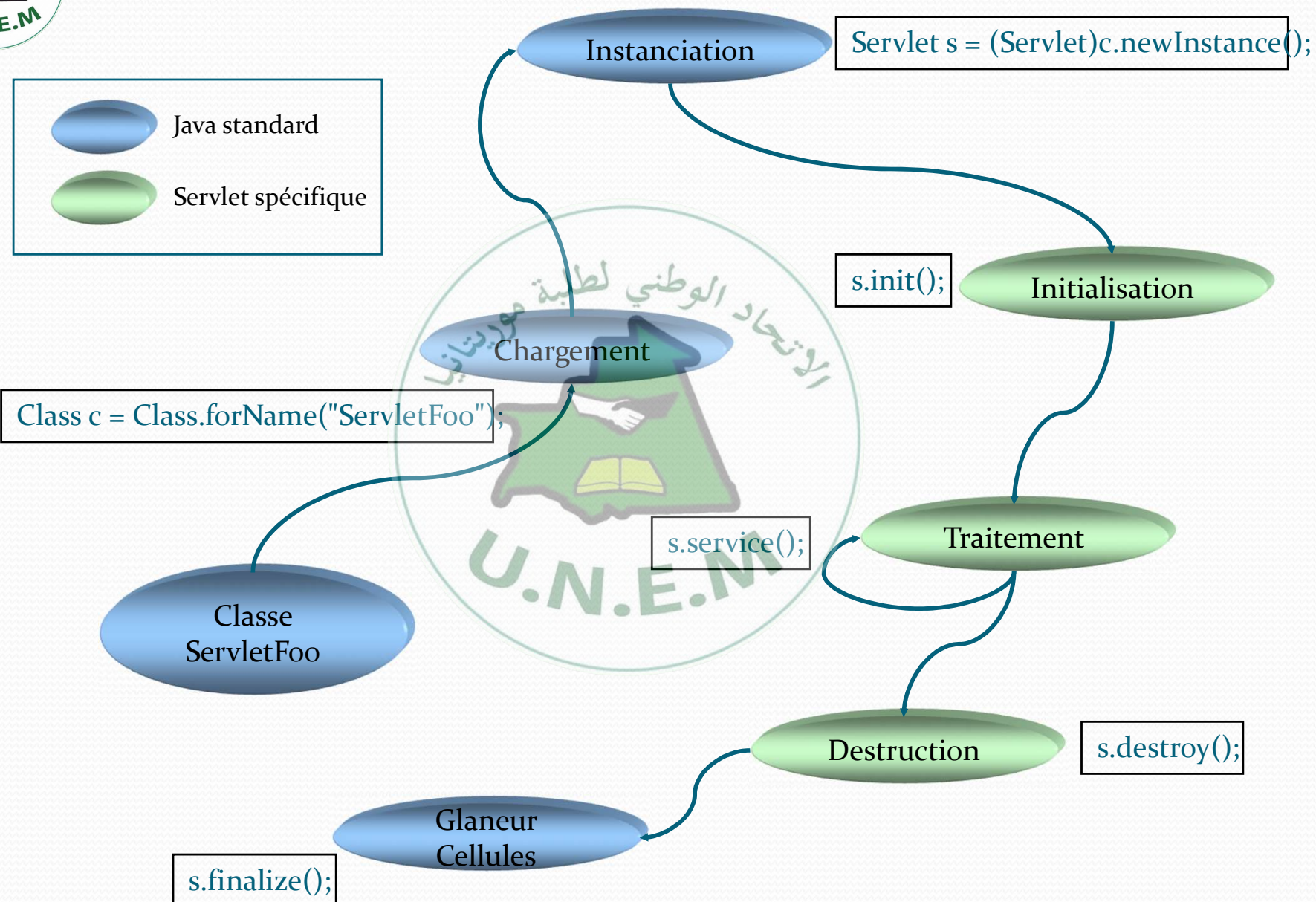
Structure de base d'une servlet

```
// importation de packages
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;

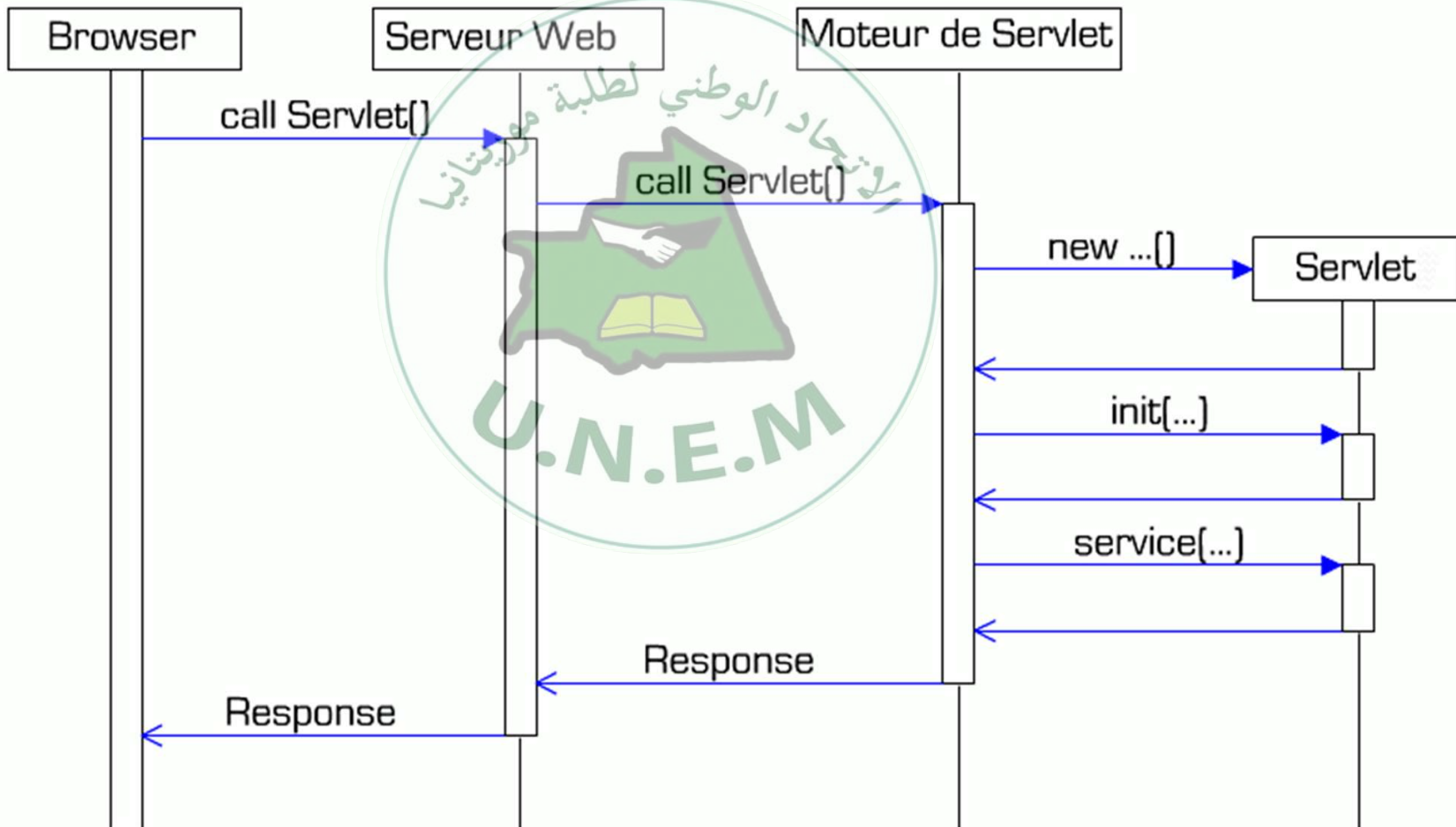
// définition de la classe
public class ServletFoo extends HttpServlet {
    public void service(HttpServletRequest request, HttpServletResponse response) {
        // positionne le type MIME pour la réponse
        response.setContentType("text/html");
        // récupère une référence vers le flux d'écriture
        PrintWriter out = response.getWriter();
        // création de la page HTML
        out.println("<HTML>")
        out.println("<HEAD><TITLE>Servlet FOO</TITLE></HEAD>");
        out.println("<BODY>");
        out.println("<H1>Voici enfin la premiere Servlet FOO</H1>");
        out.println("</BODY></HTML>");
        // toujours fermer le flux d'écriture
        out.close();
    }
}
```



cycle de vie des servlets

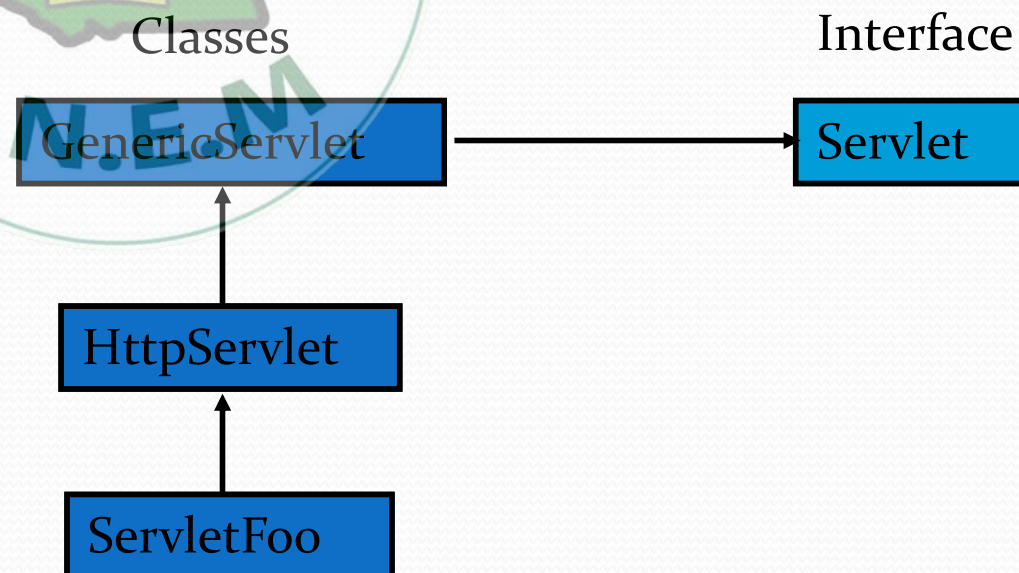


Le cycle de vie des servlets



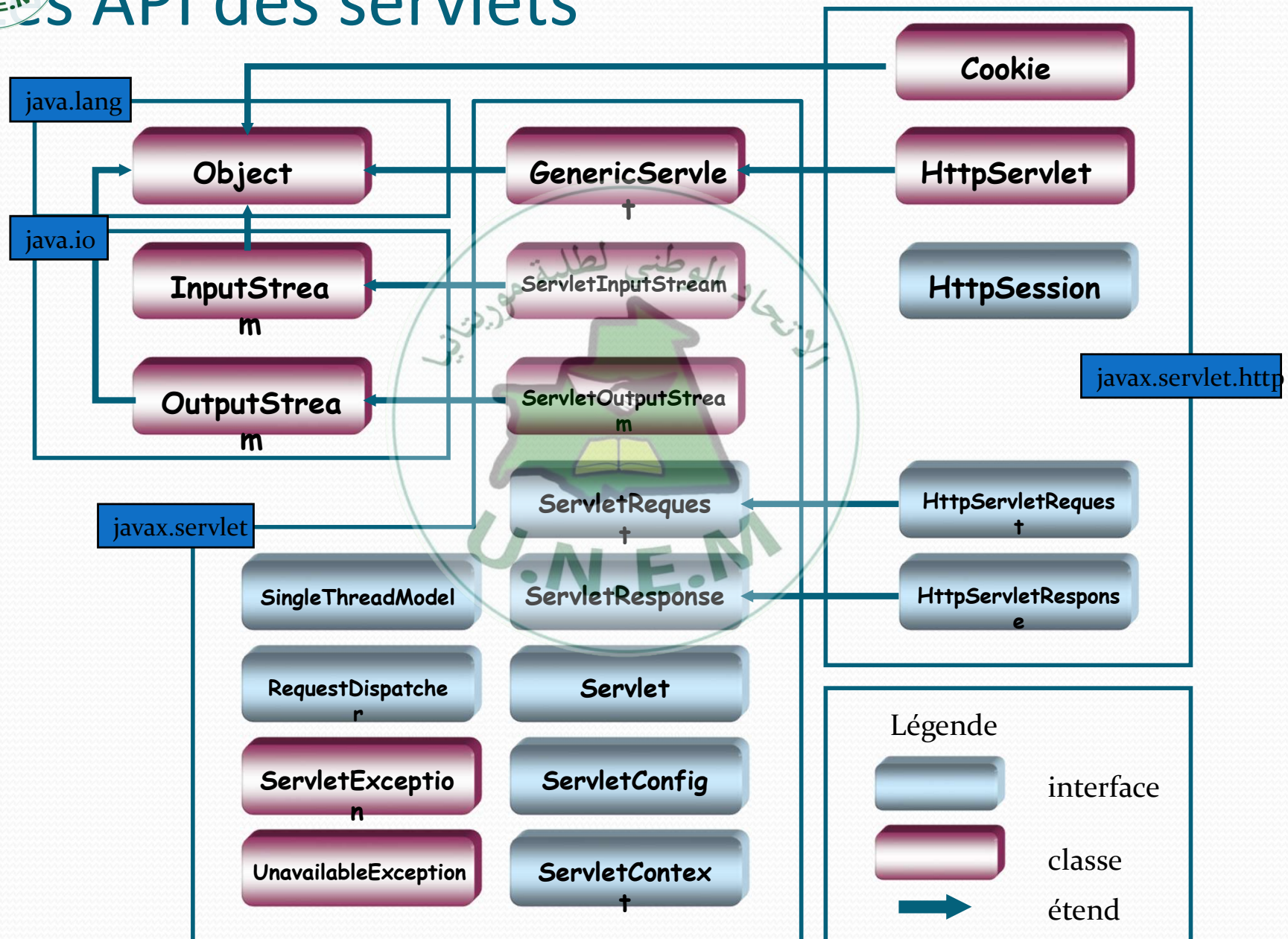
Les API des servlets

- Le package javax.servlet fournit les interfaces et les classes pour écrire des servlets génériques.
- Le package javax.servlet.http fournit les définitions pour les servlets HTTP.
- Sommairement, nous avons :





API des servlets





Les API des servlets

- Nous allons donc examiner successivement les éléments suivants :
 - L'interface `javax.servlet.Servlet` qui définit, comme nous l'avons déjà indiqué, le cycle de vie des servlets.
 - L'interface `javax.servlet.ServletConfig` pour que la servlet puisse accéder à des informations d'initialisation et de contexte d'exécution.
 - La classe `javax.servlet.GenericServlet` qui implémente l'interface "Servlet" et fournit la structure de base pour la définition de servlets.
 - La classe `javax.servlet.http.HttpServlet` pour la définition de servlets HTTP
 - L'interface `javax.servlet.ServletRequest` pour l'envoi de données vers une servlet générique
 - L'interface `javax.servlet.http.HttpServletRequest` pour l'envoi de données vers une servlet HTTP
 - L'interface `javax.servlet.ServletResponse` pour la réception de données depuis une servlet générique
 - L'interface `javax.servlet.http.HttpServletResponse` pour la réception de données depuis une servlet HTTP

L'interface Servlet

- Définit les méthodes représentant le cycle de vie d'une servlet.
 - *public void init(ServletConfig config) throws ServletException*
 - *public void service(ServletRequest rq, ServletResponse res) throws ServletException*
 - *public void destroy()*
 - *public ServletConfig getServletConfig()*
 - *public String getServletInfo()*

L'interface ServletConfig

- Définit les méthodes permettant à une servlet d'accéder aux informations de configuration initiales ainsi qu'à l'environnement d'exécution.
- Deux méthodes d'accès aux paramètres initiaux :
 - *getInitParameter (String name)*
 - *getInitParameterNames*
- Une méthode d'accès à l'environnement d'exécution de la servlet
 - *getServletContext*

L'interface ServletConfig

- ***getInitParameter***

- `public String getInitParameter(String name)`
Retourne une String qui représente la valeur d'un paramètre initial particulier. Les paramètres initiaux sont lus depuis le fichier web.xml. L'ajout d'un paramètre se fait en incluant les lignes suivantes au niveau de la balise `<servlet>`. Si le paramètre spécifié n'existe pas alors la méthode retourne la valeur null.

```
<init-param>  
    <param-name>rangeFib</param-name>  
    <param-value>3</param-value>  
</init-param>
```

L'interface ServletConfig

- ***getInitParameter***

- `public String getInitParameter(String name)`
Retourne une String qui représente la valeur d'un paramètre initial particulier. Si le paramètre spécifié n'existe pas alors la méthode retourne la valeur null.

```
public void init(ServletConfig config) throws ServletException {  
    // positionne le ServletConfig  
    super.init(config);  
    // initialise une variable d'état de la servlet  
    rangFib = Integer.parseInt(config.getInitParameter("RangFibInitial"));  
}
```

L'interface ServletConfig

- ***getInitParameterNames***

- `public Enumeration getInitParameterNames()`
Retourne une énumération qui contient sous forme de String l'ensemble des noms des paramètres initiaux. Les paramètres initiaux sont lus depuis un fichier de propriétés ou le fichier web.xml

```
public void init(ServletConfig config) throws ServletException {  
    // positionne le type MIME pour la réponse  
    rep.setContentType("text/html");  
    // récupère une référence vers le flux d'écriture  
  
    try (PrintWriter out = rep.getWriter()) {  
        // récupère le contenu du formulaire saisi par le client  
        Enumeration<String> parameters = req.getParameterNames();  
        while (parameters.hasMoreElements()) {  
            String nom = parameters.nextElement();  
            String valeur = req.getParameter(nom);  
            out.println("Nom du champ : " + nom + " valeur :  
" + valeur + "<BR>");  
        }  
    } catch (IOException e) {  
    }  
}
```

L'interface ServletConfig

- Exemple qui affiche le résultat du calcul de fibonacci dont le rang est obtenu comme paramètre initial

```
// importation de packages
import javax.servlet.*;
import java.servlet.http.*;
import java.io.*;

// définition de la classe
public class ServletFib extends HttpServlet {
    int rangFib;

    public void init(ServletConfig config) throws ServletException {
        // positionne le ServletConfig
        super.init(config);
        // initialise une variable d'état de la servlet
        rangFib = Integer.parseInt(config.getInitParameter("RangFibInitial"));
    }
    ...
}
```

L'interface ServletConfig

- la suite ...

```
public void service(HttpServletRequest req, HttpServletResponse rep) {  
    // positionne le type MIME pour la réponse  
    rep.setContentType("text/html");  
    // récupère une référence vers le flux d'écriture  
    PrintWriter out = rep.getWriter();  
    // création de la page HTML  
    out.println("<HTML>")  
    out.println("<HEAD><TITLE>Servlet FIB</TITLE></HEAD>");  
    out.println("<BODY>");  
    out.println("<H1>Fib (" + rangFib + ") = " + fib(rangFib)</H1>");  
    out.println("</BODY></HTML>");  
    // toujours fermer le flux d'écriture  
    out.close();  
}  
  
public static int fib(int n) {  
    return <le calcul de fibonacci de n;  
}  
}
```



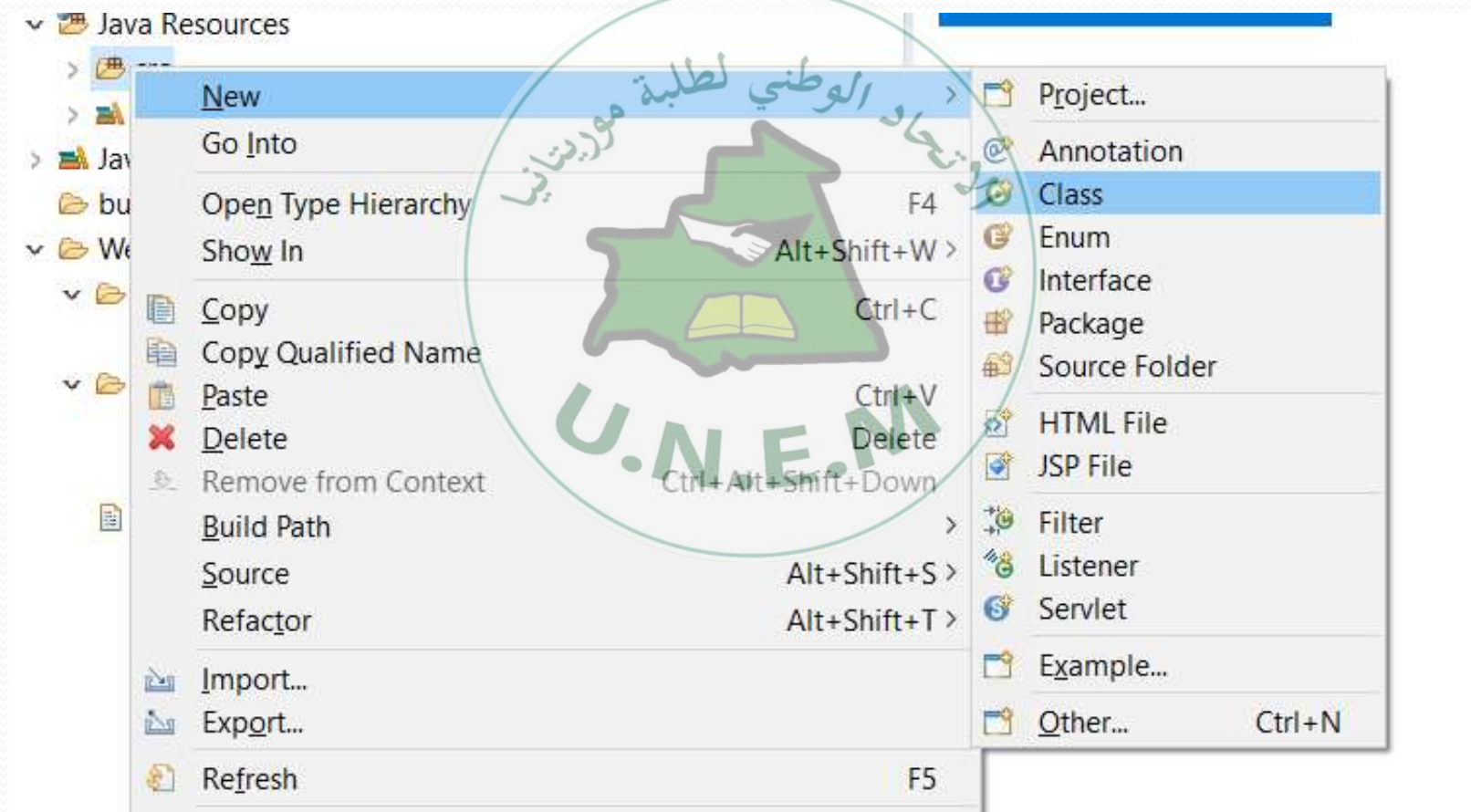
declarer une variable d'initialisation

- Aller dans web.xml au niveau de la balise **<servlet>**

```
<init-param>  
  <param-name>rangeFib</param-name>  
  <param-value>3</param-value>  
</init-param>
```



Créons une première servlet src → new → Class





Créons une première servlet

New Java Class

Java Class
Create a new Java class.

Source folder:

Package:

☐ Enclosing type:

Name:

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static

Superclass:

Interfaces:

Which method stubs would you like to create?

☐ public static void main(String[] args)
☐ Constructors from superclass
☒ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))
☐ Generate comments



code HTML permettant de faire HelloWorld

- A vous de le faire

```
import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

public class HelloWorld extends HttpServlet {

    @Override
    protected void service(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        // TODO Auto-generated method stub
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<HTML>");
        out.println("<BODY>");
        out.println("Bonjour Nomane");
        out.println("</BODY>");
        out.println("</HTML>");
        // super.service(request, response);
        out.close();
    }
}
```



5 étapes pour déployer la servlet HelloWorld

- Il s'agit d'une classe Java qui se déploie différemment qu'une simple page HTML. Il va falloir lier notre servlet à une URL.
 - Cela se fait par le biais d'un certain nombre de modifications qui doivent se faire au niveau du fichier **web.xml** qui se trouve dans **/WEB-INF (WebContent/WEB-INF)**
 - Si le fichier **web.xml** n'existe pas il faut le créer manuellement dans ce dossier



5 étapes pour déployer la servlet HelloWorld

- Le contenu du fichier web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd" id="WebApp_ID"
version="3.1">
  <display-name>NomDeVotreProjet</display-name>
  <welcome-file-list>
    <welcome-file>index.html</welcome-file>
    <welcome-file>index.htm</welcome-file>
    <welcome-file>index.jsp</welcome-file>
    <welcome-file>default.html</welcome-file>
    <welcome-file>default.htm</welcome-file>
    <welcome-file>default.jsp</welcome-file>
  </welcome-file-list>
</web-app>
```

Facultatif. On peut le supprimer



Définition de la servlet dans web.xml

```
<servlet>
  <servlet-name>Hello</servlet-name>
  <servlet-class>mr.iscae.servlets>HelloWord</servlet-class>
</servlet>
```

- La balise responsable de la définition d'une servlet se nomme logiquement **<servlet>**, et les deux balises obligatoires de cette section sont très explicites :
 - **<servlet-name>** permet de donner un nom à une servlet. C'est ensuite via ce nom qu'on fera référence à la servlet en question. Ici, j'ai nommé notre servlet **Hello**.
 - **<servlet-class>** sert à préciser le chemin de la classe de la servlet dans votre application. Ici, notre classe a bien pour nom **HelloWord** et se situe bien dans le package **mr.iscae.servlets**.

Mapping de la servlet

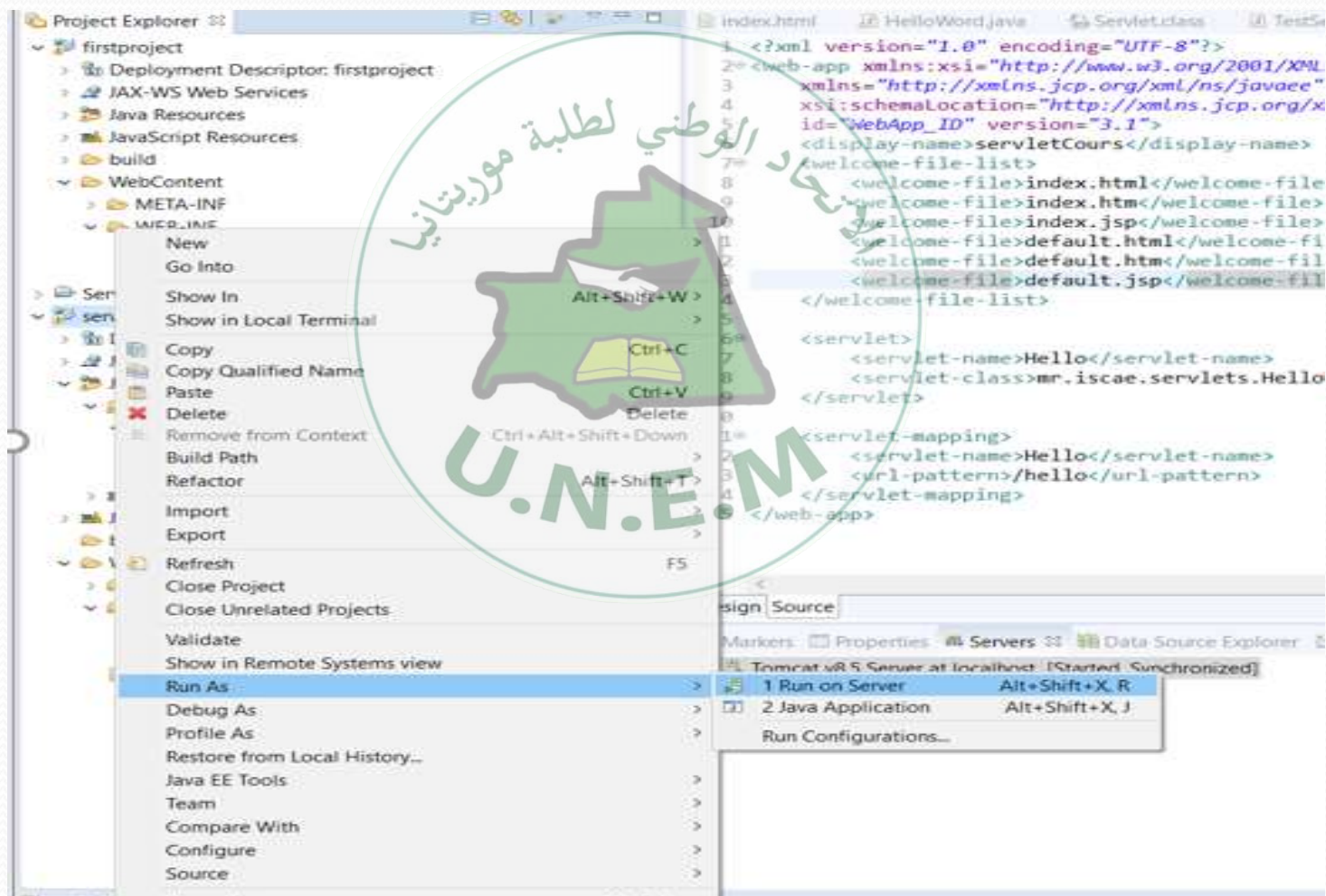
- Il faut ensuite faire correspondre notre servlet fraîchement déclarée à une URL, afin qu'elle soit joignable par les clients :

```
<servlet-mapping>  
  <servlet-name>Hello</servlet-name>  
  <url-pattern>/hello</url-pattern>  
</servlet-mapping>
```

- La balise responsable de la définition du mapping se nomme logiquement **<servlet-mapping>**, et les deux balises obligatoires de cette section sont, là encore, très explicites.
 - **<servlet-name>** permet de préciser le nom de la servlet à laquelle faire référence. **Cette information doit correspondre avec le nom défini dans la précédente déclaration de la servlet.**
 - **<url-pattern>** permet de préciser la ou les URL relatives au travers desquelles la servlet sera accessible. Ici, ça sera **/hello** !



Exécuter la servlet sur le serveur





Accéder à la Servlet

- Votre première servlet est désormais accessible sur l'url :
- Localhost:8080/NomDeVotreProjet/hello
 - Pour mon cas c'était :
<http://localhost:8080/servletCours/hello>



Les verbes HTTP

- La sémantique des méthodes HTTP
 - GET : lit les ressources, sûr, utilise un cache
 - POST : modifie une ressource
 - PUT : remplace une ressource entière
 - DELETE : supprime une ressource entière
 - HEAD

La classe HttpServlet

- Fournit le cadre pour l'implémentation des servlets reposant sur le protocole HTTP.
- On y trouve donc les méthodes représentant les traitements des différentes "méthodes HTTP". A savoir :
 - service() : pour le traitement de toute requête HTTP
 - doGet() : pour le traitement la méthode "GET"
 - doHead() : pour le traitement la méthode "HEAD"
 - doPost() : pour le traitement la méthode "POST"
 - doPut() : pour le traitement la méthode "PUT"
 - doDelete() : pour le traitement la méthode "DELETE"

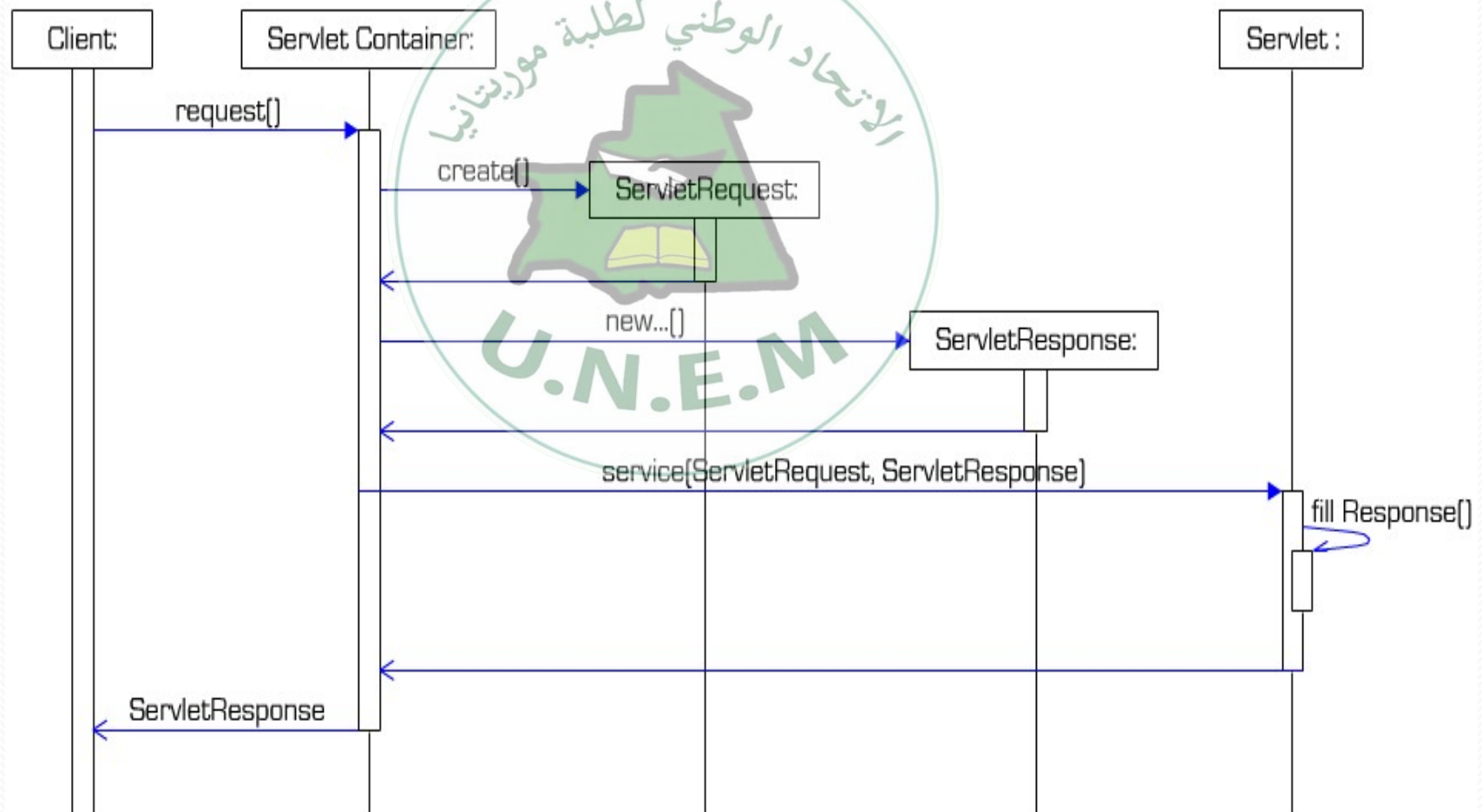


- Le communication Client serveur



Interface ServletRequest

Pour l'interaction client --> servlet





Interface ServletRequest

Pour l'interaction client --> servlet

- *public abstract String getParameter(String name)*
Pour accéder à la valeur d'un paramètre (souvent un champ d'un formulaire).

```
public void service(HttpServletRequest req, HttpServletResponse rep) {  
    // déclaration des variables locales  
    PrintWriter out;  
    String titreDuLivre;  
    // positionne le type MIME pour la réponse  
    rep.setContentType("text/html");  
    // récupère une référence vers le flux d'écriture  
    out = rep.getWriter();  
    // récupère le contenu du champ "TitreLivre" saisi par le client  
    titreDuLivre = req.getParameter("TitreLivre");  
    out.println("Titre du livre choisi : " + titreDuLivre);  
  
    // toujours fermer le flux d'écriture  
    out.close();  
}
```



Interface ServletRequest

Pour l'interaction client --> servlet

- *public abstract String[] getParameterValues(String name)*

Retourne le tableau des valeurs d'un paramètre (souvent un champ d'un formulaire).

```
public void service(HttpServletRequest req, HttpServletResponse rep) {  
    // positionne le type MIME pour la réponse  
    rep.setContentType("text/html");  
    // récupère une référence vers le flux d'écriture  
    PrintWriter out = rep.getWriter();  
    // récupère le contenu du champ "TitreLivre" saisi par le client  
    titreDuLivre = req.getParameterValues("TitreLivre")[0];  
    out.println("Titre du livre choisi : " + titreDuLivre);  
  
    // toujours fermer le flux d'écriture  
    out.close();  
}
```



Exemple d'appel avec paramètre dans la requête (*request*)

<http://localhost:8080/servletCours/hello?titreDuLivre=léon l'afiricain>

- L'ajout d'un second paramètre se fait par le biais de **&**
- Exemple :
 - [http://loclahost..../hello?param1=val1¶m2=val2](http://loclahost....hello?param1=val1¶m2=val2)



Interface ServletRequest

Pour l'interaction client --> servlet

- *public abstract Enumeration getParameterNames()*
Retourne l'énumération des noms des paramètres (i.e. l'ensemble des champs renseignés d'un formulaire).

```
@Override
public void service(HttpServletRequest req, HttpServletResponse rep) {
    // positionne le type MIME pour la réponse
    rep.setContentType("text/html");
    // récupère une référence vers le flux d'écriture

    try (PrintWriter out = rep.getWriter()) {
        // récupère le contenu du formulaire saisi par le client
        Enumeration<String> parameters = req.getParameterNames();
        while (parameters.hasMoreElements()) {
            String nom = parameters.nextElement();
            String valeur = req.getParameter(nom);
            out.println("Nom du champ : " + nom + " valeur : " + valeur + "<BR>");
        }
    } catch (IOException e) {

    }
}
```



Interface ServletRequest

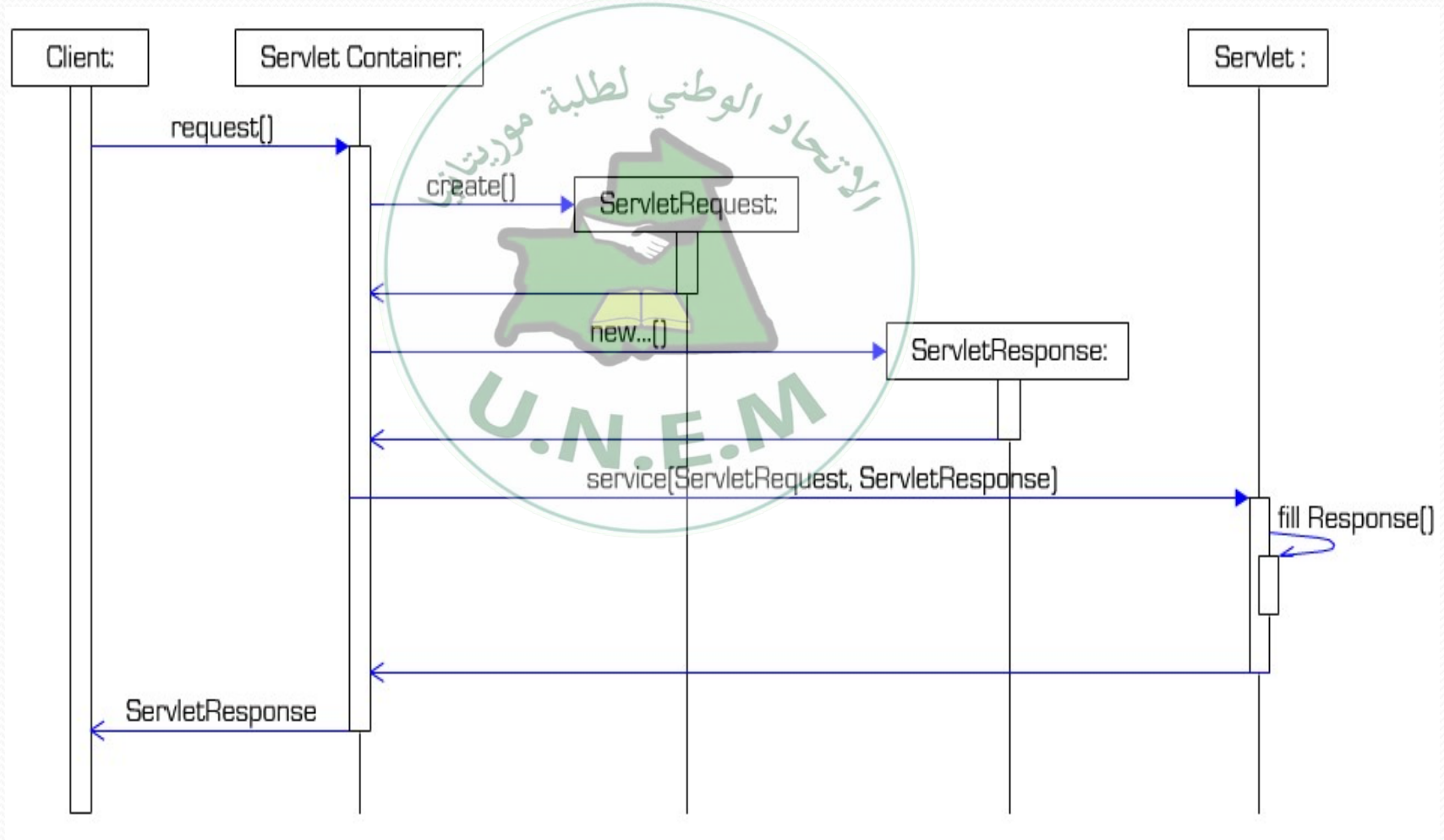
Pour l'interaction client --> servlet

- *public abstract String getProtocol()*
Retourne une String qui indique le protocole utilisé par la servlet.
Le format est le suivant : <protocole>"/"<version majeure>".<version mineure>
Pour HTTP, on aura : HTTP/1.1
- *public abstract BufferedReader getReader() throws IOException*
Retourne une BufferedReader pour analyser les données de la requête sous forme texte.
- *public abstract String getRemoteAddr()*
Retourne l'IP du client initiateur de la requête : 127.0.0.1
- *public abstract String getRemoteHost()*
Retourne le nom symbolique du client initiateur de la requête : localhost
- *public abstract String getScheme()*
Retourne le protocole utilisé par la requête, à savoir "http", "https", "ftp" ou "telnet".
- *public abstract String getServerName()*
Retourne le nom symbolique du serveur sur lequel tourne la servlet invoquée.
- *public abstract String getServerPort()*
Retourne le numéro de port qui a reçu la requête du client



Interface ServletResponse

Pour l'interaction servlet --> client





Interface ServletResponse

Pour l'interaction servlet --> client

- *public abstract int SetContentLength(int len)*
Précise la longueur de la partie données de la requête
- *public abstract String setContentType(String type)*
Précise le type MIME pour la réponse (affecte le header HTTP "Content-Type"). Les plus courants sont :
 - text/plain : Texte ASCII non formaté
 - text/html : Texte HTML formaté
 - image/gif : Image GIF
 - image/jpeg : Image JPEG
- *public abstract ServletOutputStream getOutputStream()*
throws IOException
Pour écrire des données binaires à destination du client.
- *public abstract PrintWriter getWriter()*
throws IOException
Pour écrire des données textuelles à destination du client.

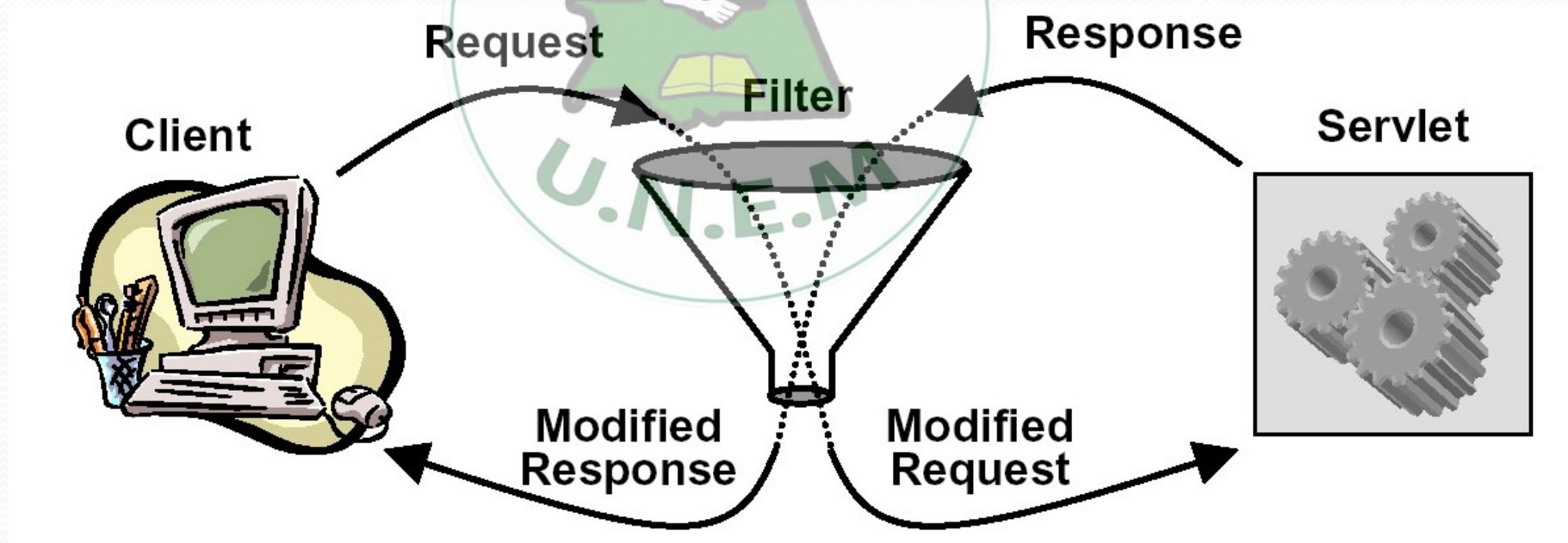


Principes du filtrage de servlets

- Décoder les requêtes du client et encoder les réponses pour le client
- Décrypter les requêtes du client et encrypter les réponses pour le client
- Authentifier les utilisateurs
- Filtrer le contenu d'une requête pour ne retenir que l'essentiel
- Convertir des images
- ...

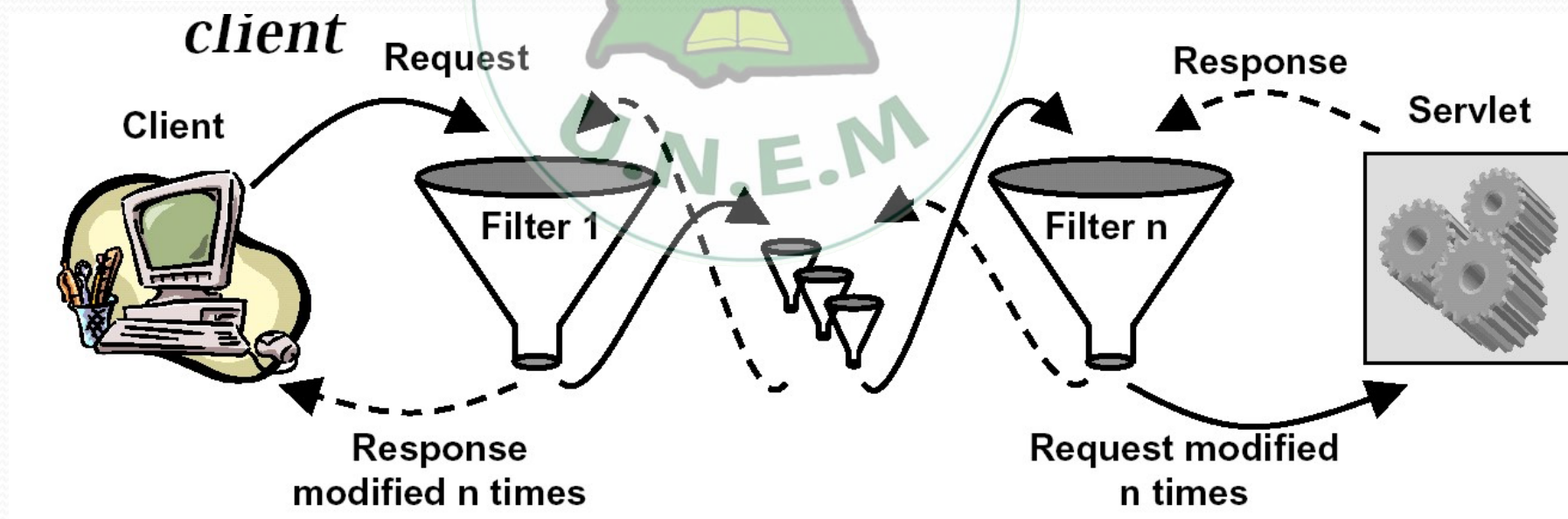
Principes du filtrage de servlets

- Les filtres de servlets peuvent être utilisés :
 - Pour changer le contenu de la requête avant de la propager vers la servlet
 - Pour changer la réponse venant de la servlet avant de la retourner au client



Principes du filtrage de servlets

- Les filtres de servlets peuvent être chaînés :
 - Pour changer le contenu de la requête en appliquant plusieurs filtres avant de la propager vers la servlet
 - Pour changer la réponse venant de la servlet en appliquant plusieurs filtres avant de la retourner au client



finir un filtre de servlet

- Un filtre de servlet est une classe qui implémente l'interface « javax.servlet.Filter » en définissant les méthodes suivantes :
 - `init(FilterConfig)` : pour initialiser le filtre
 - `doFilter()` : ce que doit faire le filtre
 - `destroy()` : pour effectuer proprement la destruction
- Les filtres de servlets peuvent accéder au contexte des servlets et aux paramètres de déploiement par l'objet « javax.servlet.FilterConfig »



finir un filtre de servlet

- Voici un exemple de filtre

```
package web.projets.bob.cryptage;
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;

public class cryptageFilter implements Filter {
    private String typeCodage = null;

    public void init(FilterConfig filterConfig) {
        typeCodage = filterConfig.getInitParameter("Codage");
    }

    public void doFilter(ServletRequest request,
        ServletResponse response, FilterChain chain)
        throws IOException, ServletException {
        Codage codage = new Codage();
        codage.decode(request, typeCodage); // crypte la requête
        chain.doFilter(request, response); // passe à la servlet
        codage.encode(response, typeCodage); // encrypte la réponse
    }
}
```

Deployment des filtres de servlet

- Les filtres fonctionnent en interceptant les requêtes destinées à une ressource ou un groupe de ressources.
- Les interceptions sont précisées dans le descripteur de déploiement de l'application web : « web.xml »
- Le déploiement d'un filtre doit préciser dans le fichier «web.xml », les éléments suivants :
 - Le nom du filtre
 - La classe du filtre
 - Les paramètres d'initialisation du filtre
 - Le mapping du filtre avec une servlet ou une URL

Déploiement des filtres de servlet



- Voici un extrait du fichier « web.xml »:

```
<filter>
  <filter-name>SessionFilter</filter-name>
  <filter-class>servlet.filtre.SessionFilter</filter-class>
  <init-param>
    <param-name>NomAtrribut</param-name>
    <param-value>NbDeVisites</param-value>
  </init-param>
</filter>

<filter-mapping>
  <filter-name>SessionFilter</filter-name>
  <url-pattern>/*</url-pattern>
</filter-mapping>
```

Jax-rs

- Les Web services

Un **service web** (ou **service de la toile**¹) est un protocole d'interface informatique de la famille des technologies web permettant la communication et l'échange de données entre applications et systèmes hétérogènes dans des environnements distribués.

- On va étudier les Web service **Rest** (un style de communication distribué)
- **Ressource & Représentation**
- Ressource = la chose « *the real thing* »
- Représentation = la réponse du serveur
"A request interacts with a resource and receives a representation."
- Utilisation de la sémantique HTTP (GET, PUT, POST, DELETE, HEAD)



Les implémentations de Jax-rs

- Jax-rs est une API et il existe plusieurs implémentations :
 - jersey, cxf, resteasy, restlet...
- Nous allons utiliser l'implémentation **JERSEY**



REST Verbs

Match HTTP semantics:

- GET : read resources, safe, cache friendly
- POST : update resources, create
- PUT : replace entire resource, atomic
- DELETE : delete entire resource, atomic
- HEAD
- PATCH

➔ Séparation de l'implémentation du serveur de la perception de la ressource par le client



- Un Web service sera servi sur une URL (@Path)
 - *Input* : Consomme des données typées (@Consumes)
 - *Output* : Produit un résultat typé (@Produces)
- Types possibles
 - Json, XML ...

Les annotations à connaître

- @Path : indique l'url sur lequel le Web service sera disponible
 - Exemple : @Path("/hello")
- @Consumes : indique le type de donnée acceptée en entrée par le Web service
 - Exemple : @Consumes("application/json")
- @Produces : indique le type de donnée produit par le Web service
 - Exemple : @Produces("application/json")



REST Service – Create (POST)

REST POST operations correspond to JPA persist operations.

@POST

@Consumes("application/json")

public void create(Customer customer) {

}



REST Service – Update (PUT)

REST PUT operations correspond to JPA merge operations.

@PUT

@Consumes({"application/xml", "application/json"})

public void update(Customer customer) {

}



5 étapes pour créer votre premier Web service

- Créer un projet de type *Web Dynamic Project*
- Ajouter les librairies Jersey dans le classpath (copier les dans le répertoire WEB-INF/lib de votre projet)
- Configurer la servlet Jersey qui va intercepter les Web services Rest. Cela se fait par l'ajout dans **web.xml** de :

```
<servlet>
  <servlet-name>jersey-servlet</servlet-name>
  <servlet-class>com.sun.jersey.spi.container.servlet.ServletContainer</servlet-class>
  <init-param>
    <param-name>com.sun.jersey.config.property.packages</param-name>
    <param-value>mr.iscae</param-value>
  </init-param>
</servlet>

<servlet-mapping>
  <servlet-name>jersey-servlet</servlet-name>
  <url-pattern>/ws-rest/*</url-pattern>
</servlet-mapping>
```

Le nom du package dans lequel se trouve vos classes

Url des web services



```
package mr.iscae;

import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.QueryParam;

@Path("/hello")
public class HelloWs {
    @GET
    public String hello(@QueryParam("name") String name) {
        return "Premier Ws cree par : " + name;
    }
}
```

- L'url : <http://localhost:9999/webServices/ws-rest/hello?name=nomane>



Deuxième exemple avec PUT

- Le but est de sauvegarder des données en mémoire et ensuite les afficher.
 - On va utiliser une Map
 - La Map stocke des clés et des valeurs
 - `Map<Integer, String> memoire = new HashMap<>();`



```
page mr.iscae;

import java.util.HashMap;
import java.util.Map;

import javax.ws.rs.GET;
import javax.ws.rs.PUT;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.QueryParam;

@Path("/mem")
@Produces("application/json")
public class MemResource {
    private Map<Integer, String> memoire = new HashMap<>();

    @GET
    public Map<Integer, String> getValue() {
        return memoire;
    }

    @PUT
    public void insert(@QueryParam("id") Integer id, @QueryParam("val")
        String val) {
        memoire.put(id, val);
    }
}
```

- Tester le service à parti d'un client HTTP
 - Postman
 - Curl
 - GET : `curl http://localhost:8080/webServices/ws-rest/mem`
 - PUT : `(curl --header "Content-Type: application/json" -X PUT "http://localhost:8080/webServices/ws-rest/mem" --data '{"id":90, "val":"lolo"})`
- `http://localhost:8080/webServices/ws-rest/mem?id=3&val=mohamed`

Un troisième exemple

- Le but de cet exemple est de passer les paramètre non pas dans l'URL mais plutôt dans le body de la requête et sous format JSON. Pour ce faire :
 - Créer une classe Data :

```
package mr.iscae;  
  
public class Data {  
    Integer id;  
    String val;  
}
```

- Suite →



- Modifier la méthode insert

```
@PUT
@Consumes(MediaType.APPLICATION_JSON)
public void insert(Data data) {
    memoire.put(data.id, data.val);
}
```